



Tecnológico de Monterrey

Aplicación de Métodos Multivariados en Ciencia de Datos

Módulo 1: Relaciones de Interdependencia

Raúl Gómez Muñoz
Blanca Rosa Ruiz Hernández

Tabla de contenidos

I Fundamentos de análisis univariado

1. Estructura y organización de datos	1
1.1. Datos ordenados	1
1.1.1. Tibbles y pipes	1
1.1.2. Ejemplo: Ordenando el conjunto de datos <code>who</code>	2
1.2. La Gramática de los Gráficos	6
1.2.1. Capas en <code>ggplot2</code>	7
1.2.2. Graficación con <code>ggplot2</code>	8
1.3. Actividad: Limpieza de datos	9
2. Evaluación de normalidad univariada	11
2.1. Los momentos estandarizados	11
2.1.1. Asimetría	12
2.1.2. Curtosis	18
2.1.3. Asimetría y curtosis muestrales	24
2.2. Gráficos QQ	36
2.3. Pruebas de normalidad	41
3. Escalamiento y transformaciones	45
3.1. Escalamiento de variables aleatorias	45
3.2. Escalamiento de muestras	50
3.3. Transformaciones	54
3.4. Optimización de normalidad (Box–Cox)	58
3.5. Actividad: Visualizando la no normalidad de muestras	61

Parte I

Fundamentos de análisis univariado

Capítulo 1

Estructura y organización de datos

Todo proyecto de ciencia de datos debe comenzar con la adquisición y organización de los conjuntos de datos requeridos. La forma en la que esta información se estructura y organiza puede tener un impacto significativo en la calidad del análisis que se desea desarrollar. En esta sección introduciremos los conceptos fundamentales del Tidyverse, un conjunto de herramientas en R que facilita la manipulación y visualización de nuestros datos siguiendo principios específicos de estructura y organización [5, 6, 7].

1.1. Datos ordenados

La idea central detrás del diseño del Tidyverse es la definición de datos ordenados [3]:

Definición 1.1.1. *Datos ordenados* es una forma de estructurar un conjunto de datos para facilitar su uso. En datos ordenados:

1. Cada variable forma una columna.
2. Cada observación forma una fila.
3. Cada tipo de unidad observacional forma una tabla.

Esta estructura facilita la manipulación, análisis y visualización de nuestra información, ya que se alinea con la forma en la que los datos se procesan típicamente en R.

1.1.1. Tibbles y pipes

En el Tidyverse, los datos suelen representarse mediante un tipo especial de data frame llamado *tibble*. Los tibbles son más amigables que los data frames tradicionales, ya que ofrecen una mejor impresión y manejo de subconjuntos de datos. Además, están diseñados para trabajar de manera fluida con el operador pipe (ya sea `|>` o bien `%>%`), que permite encadenar múltiples operaciones (conocidas como “verbos” en el contexto del Tidyverse) de manera clara y concisa. Para ilustrar estas ideas, en la siguiente sección realizaremos la limpieza del conjunto de datos `who` de la OMS, lo que nos permitirá generar un tibble que contenga esta información y que cumpla con los principios de datos ordenados. Este suele ser el primer paso en el Análisis Exploratorio de Datos (EDA) y es crucial para garantizar que los datos estén en un formato adecuado para su análisis y visualización.

1.1.2. Ejemplo: Ordenando el conjunto de datos who

Antes de comenzar, debemos cargar las librerías necesarias y el conjunto de datos `who`:

```
library(tidyverse)
library(here)
library(krulRutils)
library(magrittr)
library(kableExtra)

options(scipen = 999)
data(who)
```

Ahora podemos iniciar nuestro proceso de ordenamiento. El problema principal de este conjunto es que contiene variables (como “new_sp_m014”) que no son muy claras y que incluyen más de una unidad de información.

```
who |>
  glimpse()
```

Rows: 7,240

Columns: 60

```
$ country      <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan~
$ iso2         <chr> "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF", "AF~
$ iso3         <chr> "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AFG", "AF~
$ year         <dbl> 1980, 1981, 1982, 1983, 1984, 1985, 1986, 1987, 1988, 198~
$ new_sp_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sp_f65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_m65   <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f014  <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```


1. Estructura y organización de datos

```
$ new_sn_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_sn_f65 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m014 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_m65 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f014 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ new_ep_f65 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m014 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_m65 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f014 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f1524 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f2534 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f3544 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f4554 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f5564 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
$ newrel_f65 <dbl> NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, NA, N~
```

Es por eso que necesitamos separar estas variables e introducir nombres más claros para ellas y para sus valores asociados.

```
# Empezamos creando la variable "who_tidy"
# que contiene el conjunto de datos de la OMS ya limpio
who_tidy <- who |>
# Usamos pivot_longer para eliminar las variables que empiezan con "new"
# y usarlas como valores en su lugar
pivot_longer(
  cols = starts_with("new"),
  names_to = "key",
  values_to = "cases",
```

```
values_drop_na = TRUE
) |>
mutate(
  key = if_else(
    startsWith(key, "newrel"),
    sub("newrel", "new_rel", key),
    key
  ),
  cases = as.integer(cases)
) |>
separate(key, into = c("new", "type", "sexage"), sep = "_") |>
separate(sexage, into = c("sex", "age"), sep = 1) |>
select(-new, -iso2, -iso3) %T>%
# Finalmente, guardamos los datos ordenados en formato RDS y CSV
saveRDS(here("data", "who_tidy.rds")) |>
write_csv(here("data", "who_tidy.csv")) |>
glimpse()
```

Rows: 76,046

Columns: 6

```
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
```

Ahora queremos convertir las columnas “type”, “sex” y “age” en factores. Empezamos construyendo las siguientes tablas de correspondencia:

```
who_type_lookup_tbl <- tibble(
  code = c("ep", "rel", "sn", "sp"),
  label = c(
    "TB extrapulmonar",
    "Caso de recaída",
    "TB pulmonar baciloscopía negativa",
    "TB pulmonar baciloscopía positiva"
  )
)

who_sex_lookup_tbl <- tibble(
  code = c("f", "m"),
  label = c("Mujer", "Hombre")
)

who_age_lookup_tbl <- tibble(
```

1. Estructura y organización de datos

```
code = c(
  "014",
  "1524",
  "2534",
  "3544",
  "4554",
  "5564",
  "65"
),
label = c(
  "0-14",
  "15-24",
  "25-34",
  "35-44",
  "45-54",
  "55-64",
  "65+"
)
)
```

Utilizando estas tablas, podemos agregar ahora estas columnas a nuestro conjunto de datos:

```
# Agregamos las columnas de factores como información adicional
who_factor <- who_tidy |>
  convert_codes_to_factor(
    code_col = type,
    lookup_tbl = who_type_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Tipo
  ) |>
  convert_codes_to_factor(
    code_col = sex,
    lookup_tbl = who_sex_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Sexo
  ) |>
  convert_codes_to_factor(
    code_col = age,
    lookup_tbl = who_age_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = Edad
  ) |>
  glimpse()
```

Rows: 76,046

Columns: 9

```
$ country <chr> "Afghanistan", "Afghanistan", "Afghanistan", "Afghanistan", "A~
$ year    <dbl> 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 1997, 19~
$ type    <chr> "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "sp", "s~
$ sex     <chr> "m", "m", "m", "m", "m", "m", "m", "f", "f", "f", "f", "f", "f~
$ age     <chr> "014", "1524", "2534", "3544", "4554", "5564", "65", "014", "1~
$ cases   <int> 0, 10, 6, 3, 5, 2, 0, 5, 38, 36, 14, 8, 0, 1, 30, 129, 128, 90~
$ Tipo    <fct> TB pulmonar baciloscopía positiva, TB pulmonar baciloscopía po~
$ Sexo    <fct> Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Hombre, Mujer,~
$ Edad    <fct> 0-14, 15-24, 25-34, 35-44, 45-54, 55-64, 65+, 0-14, 15-24, 25-~
```

Finalmente, queremos usar esta información para analizar el número de casos de tuberculosis por tipo y sexo. Empezamos generando el tibble de frecuencias correspondiente:

```
who_type_sex_tbl <- who_factor |>
  count(Tipo, Sexo, wt = cases, name = "Casos")

kable(
  who_type_sex_tbl,
  booktabs = TRUE,
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 1.1: Número de casos de tuberculosis por tipo y sexo

Tipo	Sexo	Casos
TB extrapulmonar	Mujer	941880
TB extrapulmonar	Hombre	1044299
Caso de recaída	Mujer	1201596
Caso de recaída	Hombre	2018976
TB pulmonar baciloscopía negativa	Mujer	2439139
TB pulmonar baciloscopía negativa	Hombre	3840388
TB pulmonar baciloscopía positiva	Mujer	11324409
TB pulmonar baciloscopía positiva	Hombre	20586831

1.2. La Gramática de los Gráficos

El concepto de la *Gramática de los Gráficos* fue introducido por Leland Wilkinson en su libro “The Grammar of Graphics” [8], y proporciona un marco para entender cómo crear visualizaciones de forma sistemática ya que descompone el proceso de construir una gráfica en componentes como datos, estéticas, geometrías, estadísticas, coordenadas y temas. El paquete `ggplot2` [4] implementa esta gramática en R, permitiendo construir visualizaciones complejas por capas. Para más información sobre su historia y aplicaciones, consulte: [1, 2].

1. Estructura y organización de datos

1.2.1. Capas en ggplot2

En `ggplot2`, las gráficas se construyen añadiendo múltiples *capas* que definen los componentes de la visualización. Cada capa corresponde a un aspecto específico de la gráfica y, en conjunto, forman la composición completa. Estas capas incluyen:

1. **Datos:** El conjunto a visualizar (usualmente un data frame o tibble). Es la fuente de información de la gráfica.
2. **Estéticas:** Mapeos que relacionan variables con propiedades visuales, por ejemplo:
 - Posición en los ejes x e y .
 - Color, relleno.
 - Tamaño, forma.
 - Transparencia.

Estos mapeos definen *qué* datos se muestran y *cómo* se representan visualmente.

3. **Geometrías:** Objetos geométricos que muestran los datos; por ejemplo:
 - `geom_point()` para diagramas de dispersión.
 - `geom_line()` para gráficas de líneas.
 - `geom_col()` para barras.
 - `geom_histogram()` para histogramas.

La geometría controla *cómo* se dibujan los datos.

4. **Transformaciones estadísticas:** Cálculos opcionales aplicados a los datos antes de graficar, como:
 - `stat_bin()` para agrupar en histogramas.
 - `stat_smooth()` para ajustar curvas suaves.

Estos son pasos de preprocesamiento para la visualización.

5. **Escalas:** Definen cómo se traducen los valores a propiedades visuales; también controlan marcas de ejes, leyendas y guías.
6. **Coordenadas:** El sistema usado, como cartesiano (`coord_cartesian()`), polar (`coord_polar()`) o invertido (`coord_flip()`).
7. **Facetado:** Métodos para dividir los datos en subconjuntos y mostrar múltiples gráficas en una cuadrícula:
 - `facet_wrap()`
 - `facet_grid()`
8. **Etiquetas:** `labs()` agrega títulos, etiquetas de ejes y pies.
9. **Temas:** `theme()` controla la apariencia general (fuentes, fondo, rejillas).

Cada capa puede combinarse y personalizarse para construir gráficas complejas y elegantes. Esta *gramática por capas* invita a pensar en la gráfica como una composición de componentes independientes en lugar de un objeto monolítico.

1.2.2. Graficación con ggplot2

Ilustraremos estos conceptos graficando el número de casos de tuberculosis por tipo y sexo, usando el tibble de frecuencias que creamos antes.

```
# Generamos la gráfica usando ggplot2
# Cada capa corresponde a una componente de la gramática de los gráficos
who_type_sex_plot <- who_type_sex_tbl |>
  ggplot(aes(x = Tipo, y = Casos, fill = Sexo)) +
  geom_col(position = "dodge") +
  c_scale_fill("C rose", "C blue") +
  labs(
    title = "Casos de tuberculosis por tipo y sexo",
    x = "Tipo de tuberculosis",
    y = "Casos",
    fill = "Sexo"
  ) +
  theme_krul()

# Visualizamos la gráfica
who_type_sex_plot
```

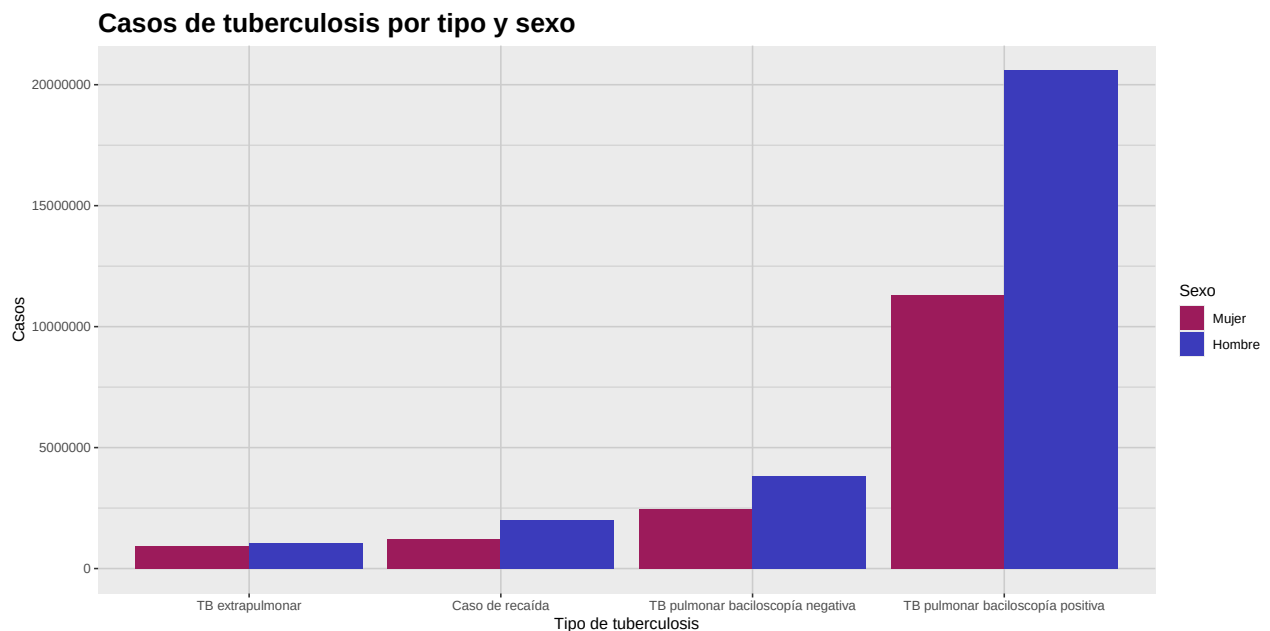


Figura 1.1: Esta gráfica muestra un análisis comparativo del número de casos de tuberculosis por tipo y sexo. Como se observa, el número de casos es consistentemente mayor en la población masculina en todos los tipos de tuberculosis.

1. Estructura y organización de datos

1.3. Actividad: Limpieza de datos

1. En la sección anterior se mostró cómo usar el operador `|>` para encadenar múltiples operaciones en el Tidyverse. En este ejercicio, muestre el resultado de todos los pasos intermedios en el proceso de ordenamiento del conjunto de datos `who` usando el operador pipe.
2. El problema con el conjunto `relig_income`, es que los distintos niveles de ingreso están representados como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la distribución del nivel de ingreso entre los distintos grupos religiosos en Estados Unidos, y genere una gráfica de barras que muestre esta información.
3. El problema con el conjunto `Billboard`, es que las semanas en que una canción apareció en la lista están representadas como columnas, lo que dificulta analizar los datos. Utilizando las técnicas vistas hasta ahora, ordene el conjunto para analizar la evolución del ranking de la canción “Who Let The Dogs Out” de “Baha Men” en el Billboard Hot 100. La visualización final debe lucir como la que se muestra abajo.

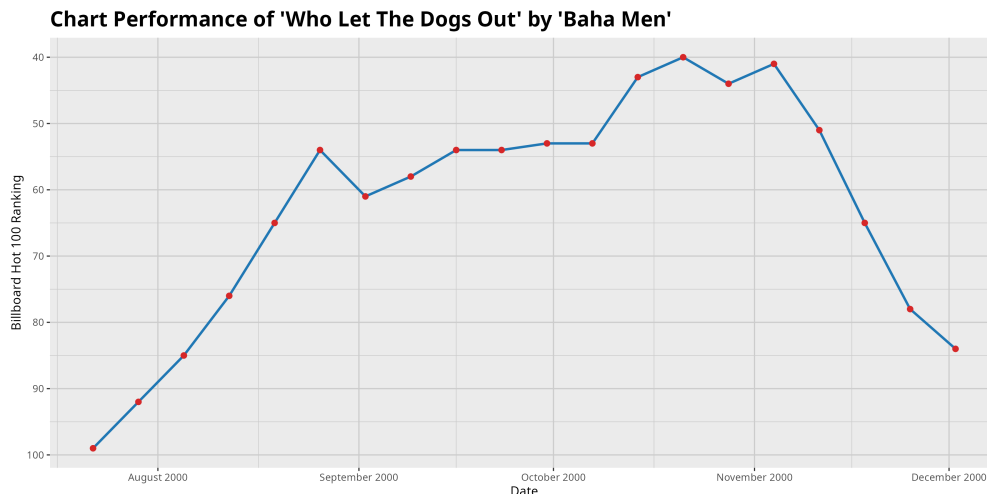


Figura 1.2: Evolución del ranking de la canción 'Who Let The Dogs Out' de 'Baha Men' en el Billboard Hot 100. La canción alcanzó el puesto 40 en la semana del 21 de octubre de 2000 y permaneció varias semanas en el top 100.

Bibliografía

- [1] Cleveland, W. S. (1993) *The Elements of Graphing Data*. Hobart Press.
- [2] Hyndman, R. J., and Athanasopoulos, G. (2021) *Forecasting: Principles and Practice* (3rd ed.). OTexts. <https://otexts.com/fpp3/>
- [3] Wickham, H. (2014) Tidy data. *Journal of Statistical Software*, 59(10), 1–23. <https://doi.org/10.18637/jss.v059.i10>
- [4] Wickham, H. (2016) *ggplot2: Elegant Graphics for Data Analysis*. Springer-Verlag.
- [5] Wickham, H., and Grolemund, G. (2017) *R for Data Science: Import, Tidy, Transform, Visualize, and Model Data*. O’Reilly Media.
- [6] Wickham, H. (2019) Functional programming with purrr. *UseR! Conference*.
- [7] Wickham, H., et al. (2019) Welcome to the tidyverse. *Journal of Open Source Software*, 4(43), 1686. <https://doi.org/10.21105/joss.01686>
- [8] Wilkinson, L. (1999) *The Grammar of Graphics*. Springer-Verlag.

Capítulo 2

Evaluación de normalidad univariada

En muchos métodos estadísticos univariados se asume que los datos siguen una distribución normal. Sin embargo, en la práctica, no todos nuestros conjuntos de datos satisfacen esta suposición. En estos casos no se justifica el uso de estos métodos y su uso indiscriminado puede llevar a conclusiones erróneas, que nos podrían llevar a tomar decisiones equivocadas bajo la ilusión de un respaldo estadístico que no existe. Es por eso que resulta fundamental contar con herramientas que nos permitan evaluar la normalidad de nuestros datos antes de aplicar cualquier método estadístico que dependa de esta suposición.

En este capítulo exploraremos algunas de estas herramientas, incluyendo las medidas de asimetría y curtosis, los gráficos QQ y algunas pruebas estadísticas para evaluar la normalidad. En el siguiente capítulo describiremos ciertas transformaciones que podemos aplicar a nuestros datos para hacerlos más cercanos a una distribución normal, en caso de que no lo sean. Pero antes de comenzar, cargaremos las librerías necesarias y definiremos algunas opciones globales para nuestro análisis.

```
library(tidyverse)
library(patchwork)
library(krnlRutils)
library(e1071)
library(latex2exp)
library(greybox)
library(kableExtra)
library(car)
options(scipen = 999)
```

2.1. Los momentos estandarizados

Como es bien sabido, cualquier distribución normal queda completamente determinada por su media y su varianza. Se sigue que todos sus momentos estandarizados de orden superior quedan fijos y se pueden utilizar para comparar otras distribuciones contra la normal. En particular, el tercer y cuarto momentos estandarizados, conocidos como asimetría y curtosis respectivamente, son de gran utilidad para describir la forma de una distribución.

2.1.1. Asimetría

Definición 2.1.1. La *asimetría* de una variable aleatoria X se define como el tercer momento estandarizado, dado por

$$\gamma_1 = \frac{\mu_3}{\sigma^3} = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

La asimetría, como su nombre lo indica, mide la falta de simetría de una distribución. Una distribución es *positivamente asimétrica* (*sesgo a la derecha*) si posee una cola más larga a la derecha, y *negativamente asimétrica* (*sesgo a la izquierda*) si la cola más larga está a la izquierda. Una distribución perfectamente simétrica tiene asimetría cero.

Para ilustrar estas ideas, introduciremos la distribución Gamma.

Definición 2.1.2. Decimos que una variable aleatoria continua X sigue una *distribución Gamma* con parámetro de forma $\alpha > 0$ y parámetro de tasa $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x; \alpha, \beta) = \begin{cases} \frac{\beta^\alpha}{\Gamma(\alpha)} x^{\alpha-1} e^{-\beta x}, & x > 0, \\ 0, & x \leq 0, \end{cases} \quad (2.1)$$

donde $\Gamma(\alpha)$ denota a la función Gamma, definida como

$$\Gamma(\alpha) = \int_0^\infty t^{\alpha-1} e^{-t} dt. \quad (2.2)$$

En este caso escribimos

$$X \sim \text{Gamma}(\alpha, \beta).$$

Observación 2.1.3. La distribución Gamma se usa frecuentemente en procesos de Poisson para modelar tiempos de espera hasta la ocurrencia de ciertos eventos. Por ejemplo, puede modelar el tiempo que pasa hasta que una compañía de seguros reciba un cierto número de reclamaciones.

La siguiente tabla resume las características poblacionales más importantes de la distribución Gamma:

```
# Crear la tabla de características de la distribución Gamma
gamma_tbl <- tibble(
  `Característica` = c("Media", "Varianza", "Asimetría"),
  `Valor` = c("$\\alpha/\\beta$", "$\\alpha/\\beta^2$", "$2/\\sqrt{\\alpha}$")
)

# Formatear y mostrar la tabla
kable(
  gamma_tbl,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  align = c("l", "c")
)
```

2. Evaluación de normalidad univariada

Tabla 2.1: Características poblacionales de la distribución Gamma con parámetros α y β .

Característica	Valor
Media	α/β
Varianza	α/β^2
Asimetría	$2/\sqrt{\alpha}$

Notemos que la asimetría siempre es positiva y tiende a cero conforme α aumenta. En otras palabras, la distribución se vuelve más simétrica conforme va aumentando el valor de α . Por otro lado, aunque no existe una fórmula cerrada para la mediana de la distribución Gamma, podemos calcularla numéricamente en R usando la función `qgamma()`. La mediana de una distribución Gamma siempre es menor que la media, lo cual es consistente con la asimetría positiva de esta distribución.

Observación 2.1.4. Más adelante definiremos la *curtosis excesiva* de una variable aleatoria, pero por el momento mencionaremos que la curtosis excesiva de la distribución Gamma está dada por $6/\alpha$. Notemos que, al igual que la asimetría, la curtosis excesiva es siempre positiva y tiende a cero conforme α aumenta, indicando que la distribución se vuelve más parecida a la normal conforme aumenta el valor de este parámetro.

Mostraremos ahora cómo graficar la PDF de una familia de distribuciones Gamma con $\beta = 1$ y distintos valores de α .

```
# Asignamos los parámetros
alpha_values <- c(1.5, 2, 3, 4, 6, 8)
beta <- 1

# Creamos una secuencia de valores para X
x_data <- seq(0, 15, length.out = 500)

# Calculamos la PDF de la familia de distribuciones Gamma
gamma_distribution_family_tbl <- expand_grid(
  x_data = x_data,
  alpha = alpha_values
) |>
  mutate(
    density_data = dgamma(x_data, shape = alpha, rate = beta),
    alpha = factor(alpha)
  )

# Generamos la gráfica de la familia
gamma_distribution_plot <- gamma_distribution_family_tbl |>
  ggplot(aes(x = x_data, y = density_data, color = alpha)) +
  geom_line(linewidth = 1) +
  scale_color_manual(
    values = unname(c_pal(
      "C red", "C yellow", "C orange",
```

```

    "C green", "C blue", "C magenta"
  )),
  labels = c(
    TeX("$\\alpha$ = 1.5"), TeX("$\\alpha$ = 2"), TeX("$\\alpha$ = 3"),
    TeX("$\\alpha$ = 4"), TeX("$\\alpha$ = 6"), TeX("$\\alpha$ = 8")
  )
) +
labs(
  title = "Familia de distribuciones Gamma con beta = 1",
  x = TeX("$X$"),
  y = "Densidad",
  color = "Parámetro de forma"
) +
theme_krul()

# Mostramos la gráfica
gamma_distribution_plot

```

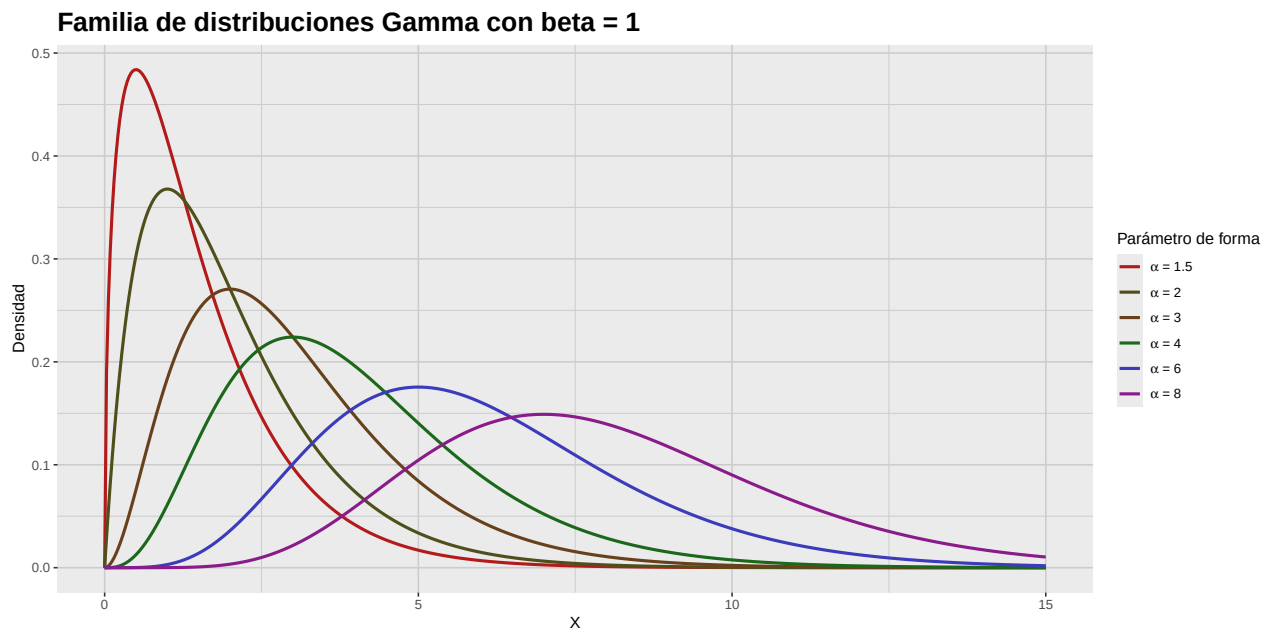


Figura 2.1: Funciones de densidad de una familia de distribuciones Gamma con $\beta = 1$ y distintos valores de α . Notemos cómo la forma de la distribución cambia conforme varía el parámetro de forma α , afectando la asimetría de la distribución. En particular, a medida que α aumenta, la distribución se vuelve más simétrica y el valor de la asimetría disminuye.

Finalmente, usaremos la distribución Gamma para generar una gráfica comparativa de una distribución con sesgo a la izquierda, una simétrica y una con sesgo a la derecha.

2. Evaluación de normalidad univariada

```
# Fijamos un valor para alpha
alpha <- 2

# Calculamos la media, la mediana y la desviación estándar
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
gamma_sd <- sqrt(alpha) / beta

# Creamos una secuencia de valores para X
x_data <- seq(0, 8, length.out = 1000)

# Calculamos la PDF de la distribución con sesgo negativo
left_skew_distribution_tbl <- tibble(
  x_data = -x_data,
  density_data = dgamma(-x_data, shape = alpha, rate = beta)
)

# Calculamos la PDF de la distribución simétrica
symmetric_distribution_tbl <- tibble(
  x_data = x_data - 4,
  density_data = dnorm(x_data)
)

# Calculamos la PDF de la distribución con sesgo positivo
right_skew_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgamma(x_data, shape = alpha, rate = beta)
)

# Generamos la gráfica de la distribución con sesgo negativo
left_skew_distribution_plot <- left_skew_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  geom_vline(
    xintercept = -gamma_mean,
    color = c_pal("C red"),
    linetype = "dashed",
    linewidth = 1
  ) +
  geom_vline(
    xintercept = -gamma_median,
    color = c_pal("C green"),
```

```
    linetype = "dashed",
    linewidth = 1
) +
annotate(
  "text",
  x = -gamma_mean,
  y = 0.5,
  label = "Media",
  color = c_pal("C red"),
  hjust = 1.1
) +
annotate(
  "text",
  x = -gamma_median,
  y = 0.5,
  label = "Mediana",
  color = c_pal("C green"),
  hjust = -0.1
) +
labs(
  title = "Sesgo a la izquierda",
  x = TeX("$X$"),
  y = "Densidad"
) +
theme_krul()

# Generamos la gráfica de la distribución simétrica
symmetric_distribution_plot <- symmetric_distribution_tbl |>
ggplot(aes(x = x_data, y = density_data)) +
geom_line(
  color = c_pal("C blue"),
  linewidth = 1
) +
geom_vline(
  xintercept = 0,
  color = c_pal("C red"),
  linetype = "dashed",
  linewidth = 1
) +
geom_vline(
  xintercept = 0,
  color = c_pal("C green"),
  linetype = "dashed",
  linewidth = 1
) +
```

2. Evaluación de normalidad univariada

```
annotate(  
  "text",  
  x = 0,  
  y = 0.5,  
  label = "Media y Mediana",  
  color = c_pal("C red"),  
  hjust = -0.1  
) +  
labs(  
  title = "Simétrica",  
  x = TeX("$X$"),  
  y = "Densidad"  
) +  
theme_krul()  
  
# Generamos la gráfica de la distribución con sesgo positivo  
right_skew_distribution_plot <- right_skew_distribution_tbl |>  
ggplot(aes(x = x_data, y = density_data)) +  
geom_line(  
  color = c_pal("C blue"),  
  linewidth = 1  
) +  
geom_vline(  
  xintercept = gamma_mean,  
  color = c_pal("C red"),  
  linetype = "dashed",  
  linewidth = 1  
) +  
geom_vline(  
  xintercept = gamma_median,  
  color = c_pal("C green"),  
  linetype = "dashed",  
  linewidth = 1  
) +  
annotate(  
  "text",  
  x = gamma_mean,  
  y = 0.5,  
  label = "Media",  
  color = c_pal("C red"),  
  hjust = -0.1  
) +  
annotate(  
  "text",  
  x = gamma_median,
```

```

y = 0.5,
label = "Mediana",
color = c_pal("C green"),
hjust = 1.1
) +
labs(
  title = "Sesgo a la derecha",
  x = TeX("$X$"),
  y = "Densidad"
) +
theme_krul()

# Combinamos las gráficas
skew_comparison_plot <- left_skew_distribution_plot +
  symmetric_distribution_plot +
  right_skew_distribution_plot +
  plot_annotation(
    title = "Comparación de distribuciones con distintos niveles de asimetría",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_comparison_plot

```

Comparación de distribuciones con distintos niveles de asimetría

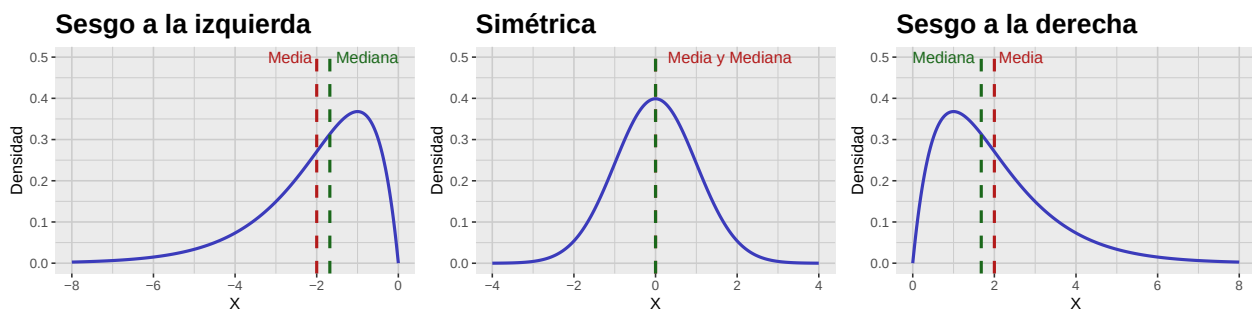


Figura 2.2: Gráfica comparativa de distribuciones con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos como la asimetría afecta la forma de la distribución y la relación entre la media y la mediana.

2.1.2. Curtosis

Definición 2.1.5. La *curtosis* de una variable aleatoria X se define como el cuarto momento estandarizado, dado por

$$\beta_2 = \frac{\mu_4}{\sigma^4} = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4},$$

donde $\mu = \mathbb{E}[X]$ es la media y $\sigma = \sqrt{\mathbb{E}[(X - \mu)^2]}$ es la desviación estándar de X .

2. Evaluación de normalidad univariada

La curtosis mide el grosor de las colas de una distribución. Una distribución con alta curtosis (*leptocúrtica*) suele tener colas más pesadas y un pico más agudo que la normal. En cambio, una distribución con baja curtosis (*platicúrtica*) suele tener colas más ligeras y un pico más plano. La curtosis de la normal es 3, por lo cual es común calcular el *exceso de curtosis* de una distribución la cual está dada por:

$$\gamma_2 = \beta_2 - 3.$$

Para ilustrar estas ideas, introduciremos la distribución normal generalizada, también conocida como la distribución de Subbotin.

Definición 2.1.6. Decimos que una variable aleatoria X sigue una *distribución normal generalizada* con parámetros de localización $\mu \in \mathbb{R}$, escala $\alpha > 0$ y forma $\beta > 0$, si su función de densidad de probabilidad (PDF) está dada por

$$f_X(x; \mu, \alpha, \beta) = \frac{\beta}{2\alpha\Gamma(1/\beta)} e^{-\left(\frac{|x-\mu|}{\alpha}\right)^\beta}, \quad x \in \mathbb{R}, \quad (2.3)$$

donde Γ nuevamente denota a la función Gamma, que definimos en (2.2). En este caso escribimos

$$X \sim \text{NormalGeneralizada}(\mu, \alpha, \beta).$$

Las características poblacionales más importantes de la distribución normal generalizada se muestran en la siguiente tabla:

```
# Crear la tabla de características de la distribución Gamma
generalized_normal_tbl <- tibble(
  `Característica` = c(
    "Media",
    "Varianza",
    "Asimetría",
    "Curtosis excesiva"
  ),
  `Valor` = c(
    "$\\mu$",
    "$\\alpha^{\\frac{2}{\\beta}} \\frac{\\Gamma(3/\\beta)}{\\Gamma(1/\\beta)}$",
    "0",
    "$\\frac{\\Gamma(5/\\beta)}{\\Gamma(1/\\beta)} \\frac{\\Gamma(3/\\beta)^2}{\\Gamma(1/\\beta)} - 3$"
  )
)

# Formatear y mostrar la tabla
kable(
  generalized_normal_tbl,
  format = "latex",
  booktabs = TRUE,
  escape = FALSE,
  align = c("l", "c")
) |>
kable_styling(latex_options = "hold_position")
```

Tabla 2.2: Características de la distribución normal generalizada con parámetros μ , α y β .

Característica	Valor
Media	μ
Varianza	$\alpha^2 \frac{\Gamma(3/\beta)}{\Gamma(1/\beta)}$
Asimetría	0
Curtosis excesiva	$\frac{\Gamma(5/\beta)\Gamma(1/\beta)}{\Gamma(3/\beta)^2} - 3$

Observación 2.1.7. Notemos que para $\beta = 1$ la distribución normal generalizada coincide con la distribución de Laplace cuya PDF está dada por

$$f_X(x; \mu, b) = \frac{1}{2b} e^{-\frac{|x-\mu|}{b}}, \quad x \in \mathbb{R}, \quad (2.4)$$

donde $b > 0$ es un parámetro de escala (que corresponde al valor de α en la normal generalizada). En este caso, la curtosis excesiva es 3, indicando colas más pesadas que la normal. Por otro lado, cuando $\beta = 2$ obtenemos la distribución normal con media μ y varianza $\alpha^2/2$. Finalmente, a medida que β tiende a infinito, la distribución normal generalizada converge a una distribución uniforme en el intervalo $(\mu - \alpha, \mu + \alpha)$, la cual tiene curtosis excesiva de -1.2 , indicando colas más ligeras que la normal.

Mostraremos ahora la gráfica de la PDF de la distribución normal generalizada para $\mu = 0$, $\alpha = 1$ y distintos valores de β , destacando sus colas y la forma del pico central.

```
# Creamos una secuencia de valores para X
x_data <- seq(-3.5, 3.5, length.out = 500)

# Seleccionamos los valores de beta a graficar
beta_values <- c(0.5, 1, 1.5, 2, 4, 8)

# Calculamos las densidades para cada valor de beta
dgnorm_distribution_family_tbl <- expand_grid(
  x_data = x_data,
  beta = beta_values
) |>
mutate(
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = beta),
  beta = factor(beta)
)

# Generamos la gráfica
dgnorm_distribution_family_plot <- dgnorm_distribution_family_tbl |>
ggplot(aes(x = x_data, y = density_data, color = beta)) +
  geom_line(linewidth = 1) +
  scale_color_manual(
    values = unname(c_pal(
```

2. Evaluación de normalidad univariada

```
"C red", "C yellow", "C orange",  
"C green", "C blue", "C magenta"  
)),  
labels = c(  
  TeX("$\\beta$ = 0.5 (Colas muy pesadas)"),  
  TeX("$\\beta$ = 1 (Laplace)"),  
  TeX("$\\beta$ = 1.5 (Colas pesadas)"),  
  TeX("$\\beta$ = 2 (Normal)"),  
  TeX("$\\beta$ = 4 (Colas ligeras)"),  
  TeX("$\\beta$ = 8 (Colas muy ligeras)")  
)  
) +  
labs(  
  title = "Distribución normal generalizada para distintos valores de beta",  
  x = TeX("$X$"),  
  y = "Densidad",  
  color = "Parámetro de forma"  
) +  
theme_krul()  
  
# Mostramos la gráfica  
dgnorm_distribution_family_plot
```

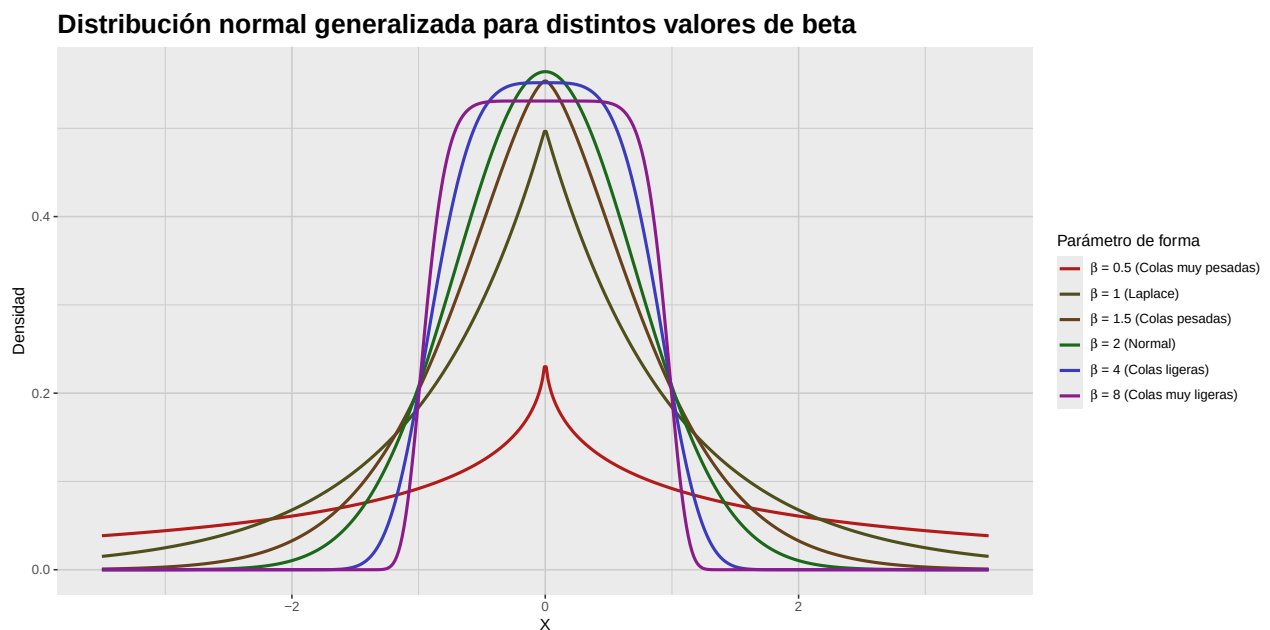


Figura 2.3: Funciones de densidad de la distribución normal generalizada para $\mu = 0$, $\alpha = 1$ y distintos valores de β . Notemos cómo cambia la forma de la distribución conforme varía el parámetro de forma β , afectando el grosor de las colas y la forma del pico central.

Finalmente, utilizaremos la distribución normal generalizada, con parámetros $\mu = 0$, $\alpha = 1$ y $\beta = 1, 2, 4$, para generar una gráfica comparativa de una distribución leptocúrtica, una mesocúrtica y una platicúrtica.

```
# Creamos una secuencia de valores para X
x_data <- seq(-2, 2, length.out = 500)

# Calculamos la PDF de la distribución leptocúrtica
leptokurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 1)
)

# Calculamos la PDF de la distribución mesocúrtica
mesokurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 2)
)

# Calculamos la PDF de la distribución platicúrtica
platykurtic_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgnorm(x_data, mu = 0, scale = 1, shape = 4)
)

# Generamos la gráfica de la distribución leptocúrtica
leptokurtic_distribution_plot <- leptokurtic_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  labs(
    title = "Leptocúrtica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Generamos la gráfica de la distribución mesocúrtica
mesokurtic_distribution_plot <- mesokurtic_distribution_tbl |>
  ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
```

2. Evaluación de normalidad univariada

```
labs(
  title = "Mesocúrtica",
  x = TeX("$X$"),
  y = "Densidad"
) +
theme_krul()

# Generamos la gráfica de la distribución platycúrtica
platykurtic_distribution_plot <- platykurtic_distribution_tbl |>
ggplot(aes(x = x_data, y = density_data)) +
  geom_line(
    color = c_pal("C blue"),
    linewidth = 1
  ) +
  labs(
    title = "Platicúrtica",
    x = TeX("$X$"),
    y = "Densidad"
  ) +
  theme_krul()

# Combinamos las gráficas
kurtosis_comparison_plot <- leptokurtic_distribution_plot +
  mesokurtic_distribution_plot +
  platykurtic_distribution_plot +
  plot_annotation(
    title = "Comparación de distribuciones con distintos niveles de curtosis",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_comparison_plot
```

Comparación de distribuciones con distintos niveles de curtosis

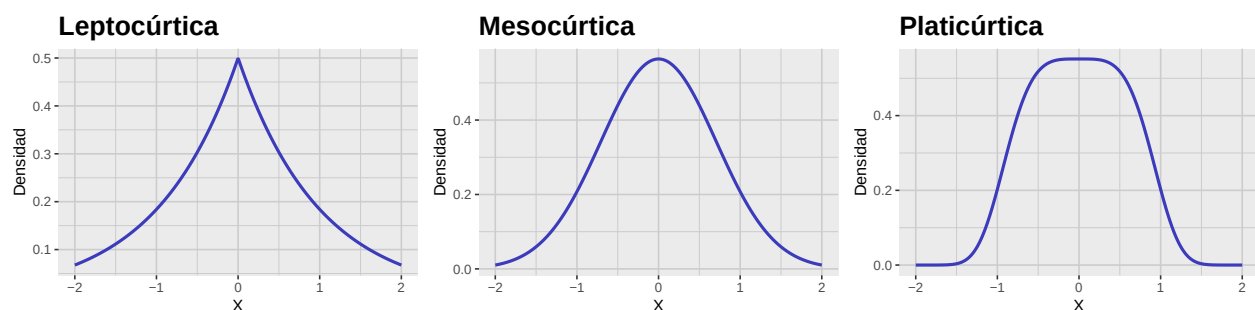


Figura 2.4: Gráfica comparativa de distribuciones leptocúrtica, mesocúrtica y platycúrtica. Notemos cómo la curtosis afecta la forma de la distribución, especialmente en las colas y el pico central.

2.1.3. Asimetría y curtosis muestrales

Hasta ahora hemos definido la asimetría y curtosis para variables aleatorias. Sin embargo, en la práctica trabajamos con muestras y no con poblaciones completas, por lo que necesitamos definir la asimetría y curtosis *muestrales*. Empecemos por recordar que, dado un *vector muestral*

$$X = \begin{bmatrix} X_1 \\ X_2 \\ \vdots \\ X_n \end{bmatrix},$$

definimos su *media muestral* como

$$\bar{X} = \frac{1}{n} \sum_{i=1}^n X_i.$$

Para la varianza muestral, tenemos dos versiones, la *varianza muestral sesgada*

$$\tilde{S}^2 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2,$$

y la *varianza muestral insesgada*

$$S^2 = \frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2.$$

En estadística, es más común utilizar la varianza muestral insesgada ya que, como su nombre lo indica, esta resulta ser un estimador insesgado de la varianza poblacional. Para definir la asimetría muestral, definiremos primero el tercer momento muestral. Igual que con la varianza muestral, existen dos versiones de esta: el *tercer momento muestral sesgado*

$$\tilde{M}_3 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^3,$$

y el *tercer momento muestral insesgado*

$$M_3 = \frac{n}{(n-1)(n-2)} \sum_{i=1}^n (X_i - \bar{X})^3.$$

A partir de aquí, hay varias posibles definiciones de la asimetría muestral que podemos considerar. Las más comunes son la *asimetría muestral de tipo 1*

$$\tilde{G}_1 = \frac{\tilde{M}_3}{\tilde{S}^3} = \frac{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^3}{\left(\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2 \right)^{3/2}} = \sqrt{n} \frac{\sum_{i=1}^n (X_i - \bar{X})^3}{\left[\sum_{i=1}^n (X_i - \bar{X})^2 \right]^{3/2}},$$

y la *asimetría muestral de tipo 2*

$$G_1 = \frac{M_3}{S^3} = \frac{\frac{n}{(n-1)(n-2)} \sum_{i=1}^n (X_i - \bar{X})^3}{\left(\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 \right)^{3/2}} = \frac{n\sqrt{n-1}}{n-2} \frac{\sum_{i=1}^n (X_i - \bar{X})^3}{\left[\sum_{i=1}^n (X_i - \bar{X})^2 \right]^{3/2}} = \frac{\sqrt{n(n-1)}}{n-2} \tilde{G}_1.$$

2. Evaluación de normalidad univariada

Aunque menos común, también podemos definir la *asimetría muestral de tipo 3* como

$$G_1^* = \frac{\widetilde{M}_3}{S^3} = \frac{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^3}{\left(\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2 \right)^{3/2}} = \frac{(n-1)^{3/2}}{n} \frac{\sum_{i=1}^n (X_i - \bar{X})^3}{\left[\sum_{i=1}^n (X_i - \bar{X})^2 \right]^{3/2}} = \left(1 - \frac{1}{n} \right)^{3/2} \widetilde{G}_1.$$

Observación 2.1.8. Es importante destacar que no existe un estimador insesgado de la asimetría poblacional. Sin embargo, todas estas versiones de la asimetría muestral son consistentes, es decir, convergen en probabilidad a la asimetría poblacional conforme el tamaño muestral n tiende a infinito. En particular, para muestras grandes, las tres versiones de la asimetría muestral son aproximadamente iguales.

En R, podemos calcular las tres versiones de la asimetría muestral utilizando la función `skewness()`, del paquete `e1071`, especificando el argumento `type` como 1, 2 o 3 según la versión que deseemos calcular. (Con la opción `type = 3`, especificada como default.) El paquete `moments` también ofrece una función similar llamada `skewness()`, pero esta solo calcula la asimetría muestral de tipo 1.

Mostraremos ahora un ejemplo de una muestra que viene de una distribución con sesgo a la izquierda, una simétrica y una con sesgo a la derecha.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(123)

# Tamaño de la muestra
n <- 3000

# Parámetros de la muestra
alpha <- 2      # forma
beta <- 1       # tasa

# Generamos las muestras
left_skew_sample <- -rgamma(n, shape = alpha, rate = beta)
symmetric_sample <- rnorm(n)
right_skew_sample <- rgamma(n, shape = alpha, rate = beta)

# Generamos un tibble para la muestra con sesgo a la izquierda
left_skew_sample_tbl <- tibble(
  sample = left_skew_sample
)

# Generamos un tibble para la muestra simétrica
symmetric_sample_tbl <- tibble(
  sample = symmetric_sample
)

# Generamos un tibble para la muestra con sesgo a la derecha
right_skew_sample_tbl <- tibble(
```

```
sample = right_skew_sample
)

# Generamos la gráfica de la muestra con sesgo a la izquierda
left_skew_sample_plot <- left_skew_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Sesgo a la izquierda",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra simétrica
symmetric_sample_plot <- symmetric_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Simétrica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra con sesgo a la derecha
right_skew_sample_plot <- right_skew_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
```


2. Evaluación de normalidad univariada

```
title = "Sesgo a la derecha",
x = "Valores muestrales",
y = "Frecuencia absoluta"
) +
theme_krul()

# Combinamos las gráficas
skew_sample_comparison_plot <- left_skew_sample_plot +
  symmetric_sample_plot +
  right_skew_sample_plot +
  plot_annotation(
    title = "Comparación de muestras con distintos niveles de asimetría",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_sample_comparison_plot
```

Comparación de muestras con distintos niveles de asimetría

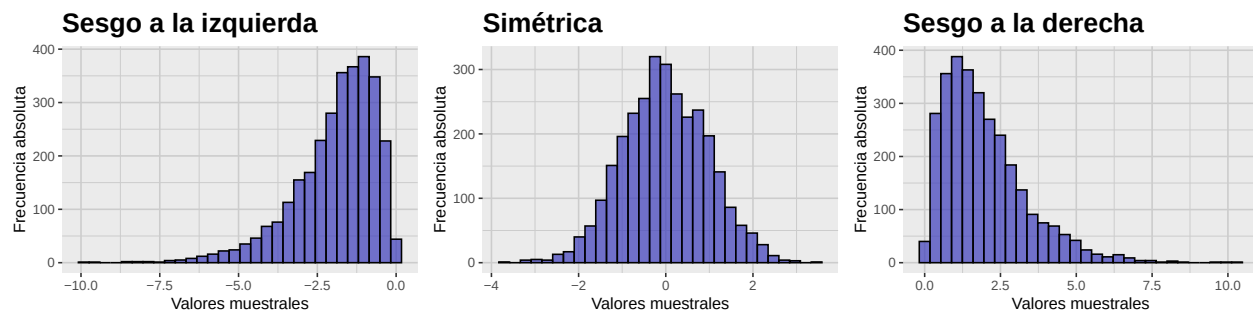


Figura 2.5: Gráfica comparativa de muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos como la asimetría nos alerta de la existencia de valores atípicos en las colas de la muestra en la dirección del sesgo.

Construiremos ahora una tabla que nos muestre los valores de la media, la mediana y los tres tipos de asimetría muestral para cada una de las muestras generadas anteriormente.

```
# Calcular media, mediana
# y los tres tipos de asimetría muestral para cada muestra
skew_sample_summary_tbl <- tibble(
  Estadística = c(
    "Media",
    "Mediana",
    "Asimetría Tipo 1",
    "Asimetría Tipo 2",
    "Asimetría Tipo 3"
  ),

```

```
`Sesgo a la izquierda` = c(
  mean(left_skew_sample),
  median(left_skew_sample),
  skewness(left_skew_sample, type = 1),
  skewness(left_skew_sample, type = 2),
  skewness(left_skew_sample, type = 3)
),
`Simétrica` = c(
  mean(symmetric_sample),
  median(symmetric_sample),
  skewness(symmetric_sample, type = 1),
  skewness(symmetric_sample, type = 2),
  skewness(symmetric_sample, type = 3)
),
`Sesgo a la derecha` = c(
  mean(right_skew_sample),
  median(right_skew_sample),
  skewness(right_skew_sample, type = 1),
  skewness(right_skew_sample, type = 2),
  skewness(right_skew_sample, type = 3)
)
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  skew_sample_summary_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.3: Tabla de media, mediana y asimetría muestral para las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos.

Estadística	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Media	-1.9375	-0.0140	1.9721
Mediana	-1.6610	-0.0452	1.6604
Asimetría Tipo 1	-1.3414	-0.0004	1.3377
Asimetría Tipo 2	-1.3421	-0.0004	1.3384
Asimetría Tipo 3	-1.3407	-0.0004	1.3370

Ahora, como sabemos que estas muestras vienen de distribuciones con media, mediana y asimetría conocida, podemos comparar los valores obtenidos para estos estadísticos muestrales con los valores de las características poblacionales que estamos estimando.

2. Evaluación de normalidad univariada

```
# Valores poblacionales conocidos
population_values_tbl <- tibble(
  `Caraterística` = c(
    "Media",
    "Mediana",
    "Asimetría"
  ),
  `Sesgo a la izquierda` = c(
    -alpha / beta,
    -qgamma(0.5, shape = alpha, rate = beta),
    -2 / sqrt(alpha)
  ),
  `Simétrica` = c(
    0,
    0,
    0
  ),
  `Sesgo a la derecha` = c(
    alpha / beta,
    qgamma(0.5, shape = alpha, rate = beta),
    2 / sqrt(alpha)
  )
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  population_values_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.4: Tabla de valores poblacionales conocidos para la media, mediana y asimetría de las distribuciones consideradas con sesgo a la izquierda, simétrica y con sesgo a la derecha.

Caraterística	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Media	-2.0000	0	2.0000
Mediana	-1.6783	0	1.6783
Asimetría	-1.4142	0	1.4142

Observación 2.1.9. Comparando estas tablas, notamos lo siguiente:

- Los valores de los distintos tipos de asimetría muestral son ligeramente distintos entre sí, pero todos indican correctamente la dirección del sesgo de la muestra. En particular, para muestras lo suficientemente grandes, todos proporcionan esencialmente la misma información.

- Los valores de estos estadísticos son ligeramente diferentes a los valores poblacionales conocidos, pero se acercan más conforme el tamaño de la muestra aumenta. Esto ilustra la propiedad de consistencia de estos estimadores, pero también ilustra la diferencia entre los valores de los estadísticos de una muestra (que suelen cambiar de una muestra a otra) y los parámetros poblacionales (que son fijos).

Para definir la curtosis muestral, empezaremos por definir el *cuarto momento muestral sesgado* como

$$\widetilde{M}_4 = \frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^4.$$

El paquete **e1071**, a través de la función **kurtosis()**, nos permite calcular tres tipos de curtosis a partir de esta definición: la *curtosis muestral de tipo 1*

$$\widetilde{G}_2 = \frac{\widetilde{M}_4}{\widetilde{S}^4} - 3 = \frac{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^4}{\left(\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^2\right)^2} - 3 = n \frac{\sum_{i=1}^n (X_i - \bar{X})^4}{\left[\sum_{i=1}^n (X_i - \bar{X})^2\right]^2} - 3,$$

la *curtosis muestral de tipo 2*

$$\begin{aligned} G_2 &= \frac{(n-1)(n+1)}{(n-2)(n-3)} \left[\widetilde{G}_2 + \frac{6}{n+1} \right] = \frac{n-1}{(n-2)(n-3)} \left[(n+1)\widetilde{G}_2 + 6 \right] \\ &= \frac{(n-1)n(n+1)}{(n-2)(n-3)} \frac{\sum_{i=1}^n (X_i - \bar{X})^4}{\left[\sum_{i=1}^n (X_i - \bar{X})^2\right]^2} - \frac{3(n-1)^2}{(n-2)(n-3)}, \end{aligned}$$

y la *curtosis muestral de tipo 3* (que es la opción por default de la función **kurtosis()** del paquete **e1071**)

$$\begin{aligned} G_2^* &= \frac{\widetilde{M}_4}{\widetilde{S}^4} - 3 = \left(1 - \frac{1}{n}\right)^2 (\widetilde{G}_2 + 3) - 3 \\ &= \frac{\frac{1}{n} \sum_{i=1}^n (X_i - \bar{X})^4}{\left(\frac{1}{n-1} \sum_{i=1}^n (X_i - \bar{X})^2\right)^2} - 3 = \frac{(n-1)^2}{n} \frac{\sum_{i=1}^n (X_i - \bar{X})^4}{\left[\sum_{i=1}^n (X_i - \bar{X})^2\right]^2} - 3. \end{aligned}$$

Notemos que todas estas definiciones estiman el exceso de curtosis. Al igual que con la asimetría muestral, no existe un estimador insesgado de la curtosis poblacional, pero todas estas versiones de la curtosis muestral son estimadores consistentes de este valor.

Mostraremos ahora un ejemplo de una muestra que viene de una distribución leptocúrtica, una mesocúrtica y una platicúrtica.

```
# Para asegurar la reproducibilidad de los resultados
set.seed(123)

# Tamaño de la muestra
n <- 3000

# Parámetros de la muestra
mu <- 0 # localización
```

2. Evaluación de normalidad univariada

```
alpha <- 1          # escala
beta_leptokurtic <- 1  # forma leptocúrtica
beta_mesokurtic <- 2   # forma mesocúrtica
beta_platykurtic <- 4   # forma platicúrtica

# Generamos la muestra leptocúrtica
leptokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_leptokurtic
)

# Generamos la muestra mesocúrtica
mesokurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_mesokurtic
)

# Generamos la muestra platicúrtica
platykurtic_sample <- rgnorm(
  n = n,
  mu = mu,
  scale = alpha,
  shape = beta_platykurtic
)

# Generamos un tibble para la muestra leptocúrtica
leptokurtic_sample_tbl <- tibble(
  sample = leptokurtic_sample
)

# Generamos un tibble para la muestra mesocúrtica
mesokurtic_sample_tbl <- tibble(
  sample = mesokurtic_sample
)

# Generamos un tibble para la muestra platicúrtica
platykurtic_sample_tbl <- tibble(
  sample = platykurtic_sample
)

# Generamos la gráfica de la muestra leptocúrtica
```

```
leptokurtic_sample_plot <- leptokurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Leptocúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra mesocúrtica
mesokurtic_sample_plot <- mesokurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Mesocúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
  theme_krul()

# Generamos la gráfica de la muestra platicúrtica
platykurtic_sample_plot <- platykurtic_sample_tbl |>
  ggplot(aes(x = sample)) +
  geom_histogram(
    bins = 30,
    fill = c_pal("C blue"),
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Platicúrtica",
    x = "Valores muestrales",
    y = "Frecuencia absoluta"
  ) +
```

2. Evaluación de normalidad univariada

```
theme_krul()

# Combinamos las gráficas
kurtosis_sample_comparison_plot <- leptokurtic_sample_plot +
  mesokurtic_sample_plot +
  platykurtic_sample_plot +
  plot_annotation(
    title = "Comparación de muestras con distintos niveles de curtosis",
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_sample_comparison_plot
```

Comparación de muestras con distintos niveles de curtosis

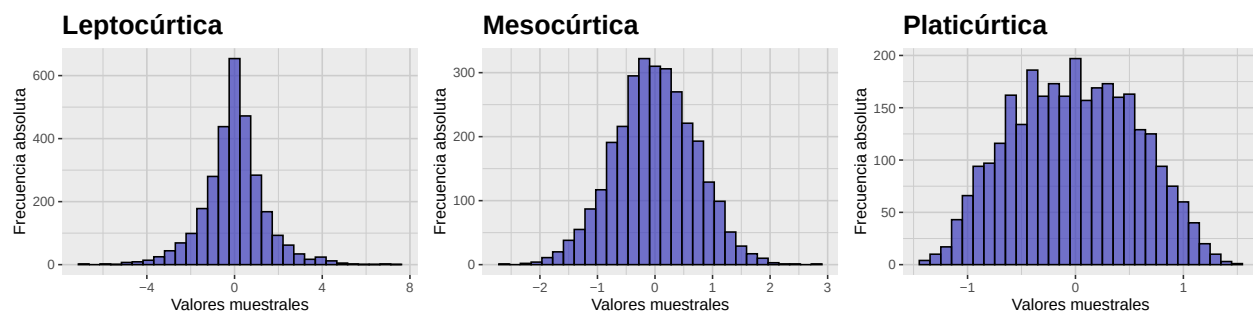


Figura 2.6: Gráfica comparativa de muestras leptocúrtica, mesocúrtica y platicúrtica. Notemos como las muestras con mayor curtosis tienden a tener más valores atípicos en las colas, mientras que las muestras con menor curtosis tienen colas más ligeras.

Construiremos ahora una tabla que nos muestre los valores de la media, la varianza y los tres tipos de curtosis muestral para cada una de las muestras generadas anteriormente.

```
# Calcular media, varianza
# y los tres tipos de curtosis muestral para cada muestra
kurtosis_sample_summary_tbl <- tibble(
  Estadística = c(
    "Media",
    "Varianza",
    "Curtosis Tipo 1",
    "Curtosis Tipo 2",
    "Curtosis Tipo 3"
  ),
  `Leptocúrtica` = c(
    mean(leptokurtic_sample),
    var(leptokurtic_sample),
    kurtosis(leptokurtic_sample, type = 1),
```

```
kurtosis(leptokurtic_sample, type = 2),
kurtosis(leptokurtic_sample, type = 3)
),
`Mesocúrtica` = c(
  mean(mesokurtic_sample),
  var(mesokurtic_sample),
  kurtosis(mesokurtic_sample, type = 1),
  kurtosis(mesokurtic_sample, type = 2),
  kurtosis(mesokurtic_sample, type = 3)
),
`Platicúrtica` = c(
  mean(platykurtic_sample),
  var(platykurtic_sample),
  kurtosis(platykurtic_sample, type = 1),
  kurtosis(platykurtic_sample, type = 2),
  kurtosis(platykurtic_sample, type = 3)
)
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  kurtosis_sample_summary_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.5: Tabla de media, varianza y curtosis muestral para las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos.

Estadística	Leptocúrtica	Mesocúrtica	Platicúrtica
Media	-0.0094	-0.0027	-0.0111
Varianza	1.9775	0.4935	0.3281
Curtosis Tipo 1	2.5569	0.0291	-0.7679
Curtosis Tipo 2	2.5631	0.0311	-0.7672
Curtosis Tipo 3	2.5532	0.0271	-0.7694

Ahora, como sabemos que estas muestras vienen de distribuciones con media, varianza y curtosis conocida, podemos comparar los valores obtenidos para estos estadísticos muestrales con los valores de las características poblacionales que estamos estimando.

```
# Valores poblacionales conocidos
population_kurtosis_values_tbl <- tibble(
  `Caraterística` = c(
    "Media",
```


2. Evaluación de normalidad univariada

```
"Varianza",
"Curtosis excesiva"
),
`Leptocúrtica` = c(
  mu,
  alpha^2 * gamma(3 / beta_leptokurtic) / gamma(1 / beta_leptokurtic),
  gamma(5 / beta_leptokurtic) * gamma(1 / beta_leptokurtic) /
  (gamma(3 / beta_leptokurtic))^2 - 3
),
`Mesocúrtica` = c(
  mu,
  alpha^2 * gamma(3 / beta_mesokurtic) / gamma(1 / beta_mesokurtic),
  gamma(5 / beta_mesokurtic) * gamma(1 / beta_mesokurtic) /
  (gamma(3 / beta_mesokurtic))^2 - 3
),
`Platicúrtica` = c(
  mu,
  alpha^2 * gamma(3 / beta_platykurtic) / gamma(1 / beta_platykurtic),
  gamma(5 / beta_platykurtic) * gamma(1 / beta_platykurtic) /
  (gamma(3 / beta_platykurtic))^2 - 3
)
)

# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  population_kurtosis_values_tbl,
  booktabs = TRUE,
  digits = 4,
  align = c("l", "c", "c", "c")
)
```

Tabla 2.6: Tabla de valores poblacionales conocidos para la media, varianza y curtosis de las distribuciones leptocúrtica, mesocúrtica y platicúrtica consideradas.

Caraterística	Leptocúrtica	Mesocúrtica	Platicúrtica
Media	0	0.0	0.0000
Varianza	2	0.5	0.3380
Curtosis excesiva	3	0.0	-0.8116

Nuevamente notamos que los valores de los distintos tipos de curtosis muestral son ligeramente diferentes entre sí, pero todos indican correctamente el nivel de curtosis de la distribución de la cual se extrajo la muestra. Además, los valores de estos estadísticos son ligeramente diferentes a los valores poblacionales conocidos, pero se acercan más conforme el tamaño de la muestra aumenta, ilustrando que estos estimadores son, efectivamente, consistentes.

2.2. Gráficos QQ

Los gráficos QQ (quantile–quantile) son una herramienta que nos permite comparar la distribución de un conjunto de datos dado contra una distribución teórica. Cuando tomamos esta distribución teórica como la normal estándar, estos gráficos nos permiten evaluar de manera visual la normalidad de nuestros datos.

Para construir un gráfico QQ, debemos seguir los siguientes pasos:

1. **Ordenar los datos muestrales.** Coloque los datos en orden creciente:

$$X_{(1)} \leq X_{(2)} \leq \cdots \leq X_{(n)}.$$

2. **Calcular las probabilidades a considerar.** Para cada valor ordenado $X_{(i)}$, calcule la probabilidad correspondiente

$$P_i = \frac{2i - 1}{2n}, \quad i = 1, 2, \dots, n.$$

3. **Obtener los cuantiles teóricos de la normal.** Usando la función cuantil de la normal estándar, podemos obtener los cuantiles teóricos correspondientes a las probabilidades calculadas en el paso anterior:

$$Q_i = \Phi^{-1}(P_i),$$

donde Φ^{-1} es la función cuantil de la normal estándar.

4. **Generar el gráfico.** Una vez obtenidos estos valores, debemos graficar los pares $(Q_i, X_{(i)})$ para $i = 1, 2, \dots, n$, en un mismo plano cartesiano. También es común agregar una recta de referencia que pase por el primer y tercer cuartil de los datos muestrales.

En general, entre más cerca se encuentren los puntos del gráfico QQ a la recta de referencia, más evidencia tendremos de que los datos muestrales provienen de una distribución normal. Por el contrario, si los puntos se alejan significativamente de esta recta, tendremos evidencia en contra de la normalidad de nuestros datos.

Para ilustrar estas ideas, consideraremos los histogramas y los gráficos QQ de las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos en la sección anterior.

```
# Generamos el gráfico QQ para la muestra con sesgo a la izquierda
left_skew_qq_plot <- left_skew_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
    color = c_pal("C magenta"),
    linewidth = 0.5
  ) +
  geom_qq(
    color = c_pal("C blue"),
    size = 0.3
  ) +
  labs(
```

2. Evaluación de normalidad univariada

```
x = "Cuantiles teóricos",
y = "Cuantiles muestrales"
) +
theme_krul()

# Generamos el gráfico QQ para la muestra simétrica
symmetric_qq_plot <- symmetric_sample_tbl |>
ggplot(aes(sample = sample)) +
geom_qq_line(
  color = c_pal("C magenta"),
  linewidth = 0.5
) +
geom_qq(
  color = c_pal("C blue"),
  size = 0.3
) +
labs(
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

# Generamos el gráfico QQ para la muestra con sesgo a la derecha
right_skew_qq_plot <- right_skew_sample_tbl |>
ggplot(aes(sample = sample)) +
geom_qq_line(
  color = c_pal("C magenta"),
  linewidth = 0.5
) +
geom_qq(
  color = c_pal("C blue"),
  size = 0.3
) +
labs(
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

# Combinamos los gráficos QQ
skew_sample_qq_comparison_plot <- left_skew_qq_plot +
  symmetric_qq_plot +
  right_skew_qq_plot

# Combinamos los gráficos QQ con los histogramas
```

```
skew_hist_qq_comparison_plot <- skew_sample_comparison_plot /
  skew_sample_qq_comparison_plot +
  plot_annotation(
    title = paste0(
      "Comparación de histogramas y gráficos QQ\n",
      "de muestras con distintos niveles de asimetría"
    ),
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
skew_hist_qq_comparison_plot
```

Comparación de histogramas y gráficos QQ de muestras con distintos niveles de asimetría

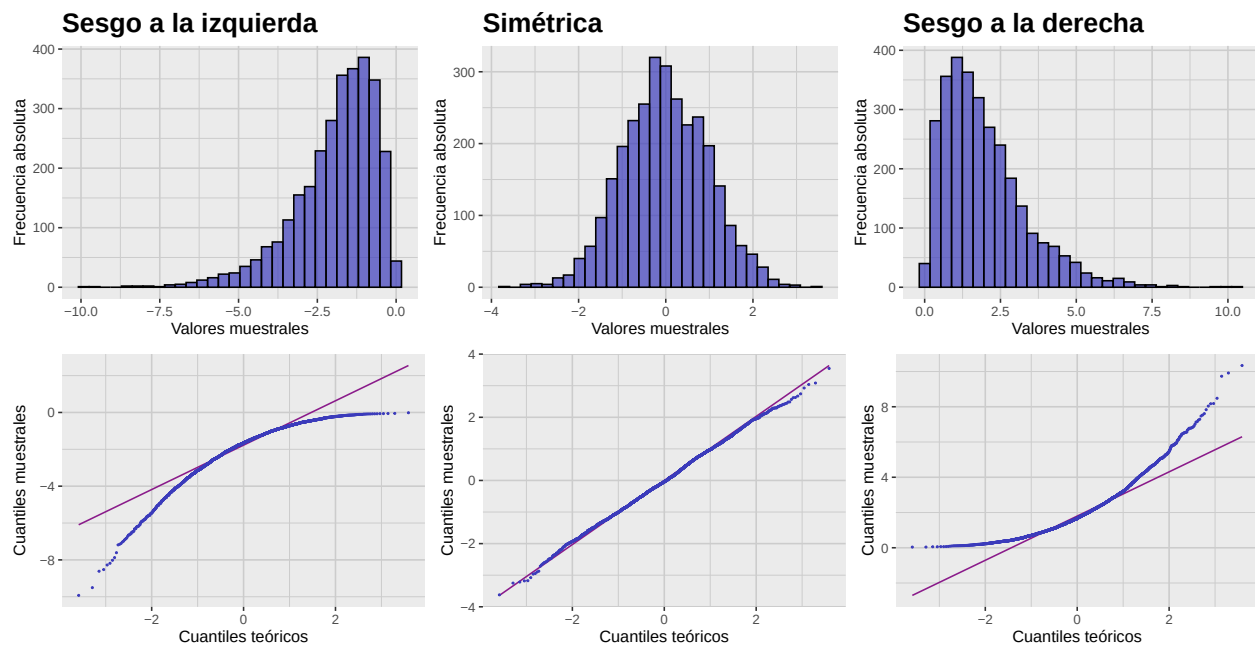


Figura 2.7: Gráfica comparativa de histogramas y gráficos QQ de muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha. Notemos cómo los gráficos QQ nos permiten evaluar visualmente la normalidad de las muestras, mostrando desviaciones significativas de la línea de referencia en las muestras sesgadas.

Consideraremos también los histogramas y los gráficos QQ de las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos en la sección anterior.

```
# Generamos el gráfico QQ para la muestra leptocúrtica
leptokurtic_qq_plot <- leptokurtic_sample_tbl |>
  ggplot(aes(sample = sample)) +
  geom_qq_line(
```

2. Evaluación de normalidad univariada

```
    color = c_pal("C magenta"),
    linewidth = 0.5
) +
geom_qq(
  color = c_pal("C blue"),
  size = 0.3
) +
labs(
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

# Generamos el gráfico QQ para la muestra mesocúrtica
mesokurtic_qq_plot <- mesokurtic_sample_tbl |>
ggplot(aes(sample = sample)) +
geom_qq_line(
  color = c_pal("C magenta"),
  linewidth = 0.5
) +
geom_qq(
  color = c_pal("C blue"),
  size = 0.3
) +
labs(
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

# Generamos el gráfico QQ para la muestra platicúrtica
platykurtic_qq_plot <- platykurtic_sample_tbl |>
ggplot(aes(sample = sample)) +
geom_qq_line(
  color = c_pal("C magenta"),
  linewidth = 0.5
) +
geom_qq(
  color = c_pal("C blue"),
  size = 0.3
) +
labs(
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
```

```
theme_krul()

# Combinamos los gráficos QQ
kurtosis_sample_qq_comparison_plot <- leptokurtic_qq_plot +
  mesokurtic_qq_plot +
  platykurtic_qq_plot

# Combinamos los gráficos QQ con los histogramas
kurtosis_hist_qq_comparison_plot <- kurtosis_sample_comparison_plot /
  kurtosis_sample_qq_comparison_plot +
  plot_annotation(
    title = paste0(
      "Comparación de histogramas y gráficos QQ\n",
      "de muestras con distintos niveles de curtosis"
    ),
    theme = theme(plot.title = element_text(size = 24, face = "bold"))
  )

# Mostramos la gráfica
kurtosis_hist_qq_comparison_plot
```

Comparación de histogramas y gráficos QQ de muestras con distintos niveles de curtosis

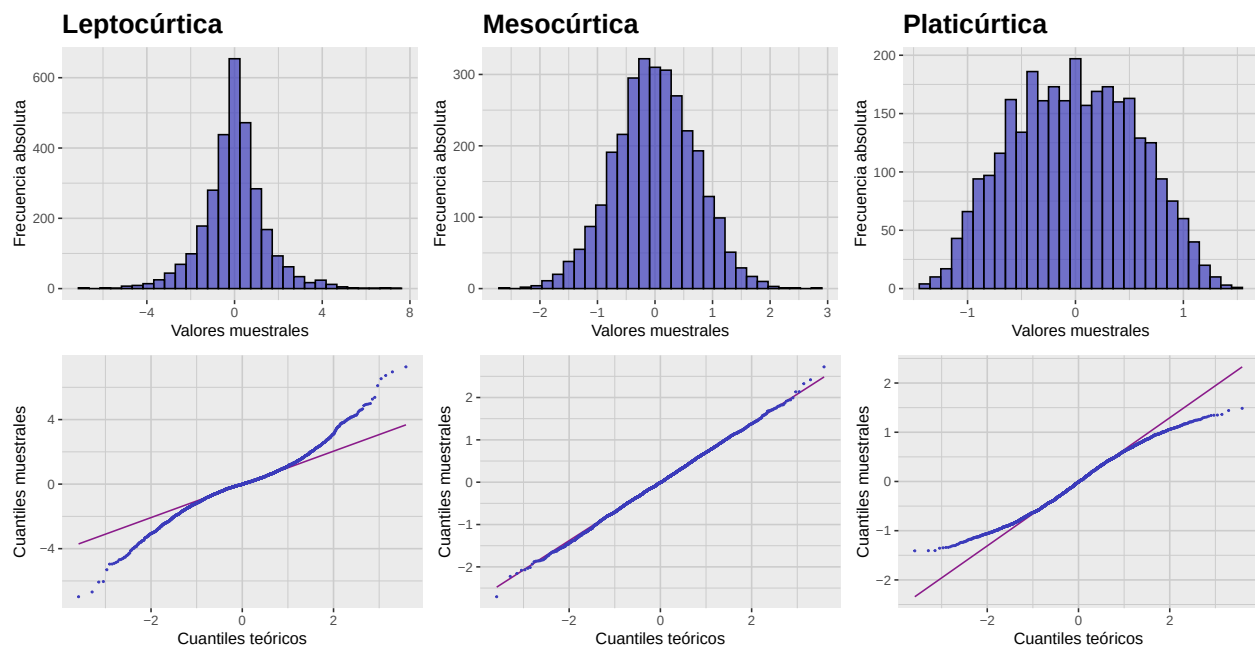


Figura 2.8: Gráfica comparativa de histogramas y gráficos QQ de muestras leptocúrtica, mesocúrtica y platicúrtica. Observemos cómo los gráficos QQ reflejan las diferencias en la curtosis de las muestras, con desviaciones notables de la línea de referencia en las muestras leptocúrtica y platicúrtica.

2. Evaluación de normalidad univariada

2.3. Pruebas de normalidad

Aunque los gráficos QQ son una buena forma de evaluar visualmente la normalidad de nuestros datos, muchas veces es necesario contar con pruebas estadísticas más objetivas y que nos permitan determinar de una manera más precisa si estos datos siguen o no una distribución normal. Algunas de las pruebas más usadas para este propósito son las siguientes:

1. **Shapiro–Wilk.** Evalúa que tan bien los datos ordenados de la muestra se ajustan a los cuantiles esperados de una distribución normal. Es muy usada para muestras pequeñas (menores a 50) pero puede ser demasiado sensible en muestras grandes. En R, se puede llamar utilizando la función `shapiro.test()`.
2. **Kolmogorov–Smirnov.** Compara la función de distribución empírica con la acumulada de la normal. Es no paramétrica y aplicable a cualquier tamaño, pero menos potente que Shapiro–Wilk en muestras pequeñas; sensible a parámetros de escala y localización. Para llamar a esta función en R, se puede usar el comando `ks.test()`.
3. **Anderson–Darling.** Similar a Kolmogorov–Smirnov, pero da mayor peso a las colas. Potente en muestras pequeñas y ampliamente utilizada. Para utilizar esta prueba en R, se puede usar la función `ad.test()` del paquete `nortest`.
4. **Jarque–Bera.** Basada en la asimetría y curtosis de los datos. Potente en muestras grandes; en muestras pequeñas puede ser demasiado sensible a leves desvíos. En R, se puede llamar utilizando la función `jarque.bera.test()` del paquete `tseries`.

Aplicamos estas pruebas a las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos anteriormente.

```
# Generamos la tabla de resultados de las pruebas de normalidad
skew_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = left_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la izquierda"
  ),
  compute_normality_pvalues(
    data = symmetric_sample_tbl,
    value_col = sample,
    sample_name = "Simétrica"
  ),
  compute_normality_pvalues(
    data = right_skew_sample_tbl,
    value_col = sample,
    sample_name = "Sesgo a la derecha"
  )
) |>
  reduce(full_join, by = "Prueba")

# Formatear y mostrar la tabla redondeando a 4 decimales
```

```
kable(
  skew_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(skew_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.7: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras con sesgo a la izquierda, simétrica y con sesgo a la derecha que generamos.

Prueba	Sesgo a la izquierda	Simétrica	Sesgo a la derecha
Shapiro-Wilk	0	0.3335	0
Kolmogorov-Smirnov	0	0.5069	0
Anderson-Darling	0	0.0894	0
Jarque-Bera	0	0.8762	0

Cabe mencionar que todas estas pruebas suelen rechazar la hipótesis de normalidad para muestras muy grandes, incluso si la desviación de la normalidad es mínima. Por lo tanto, es importante complementar los resultados de estas pruebas con gráficos QQ y análisis adicionales para tomar decisiones informadas sobre la normalidad de los datos.

Ahora hacemos lo mismo para las muestras leptocúrtica, mesocúrtica y platicúrtica.

```
# Generamos la tabla de resultados de las pruebas de normalidad
kurtosis_normality_results_tbl <- list(
  compute_normality_pvalues(
    data = leptokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Leptocúrtica"
  ),
  compute_normality_pvalues(
    data = mesokurtic_sample_tbl,
    value_col = sample,
    sample_name = "Mesocúrtica"
  ),
  compute_normality_pvalues(
    data = platykurtic_sample_tbl,
    value_col = sample,
    sample_name = "Platicúrtica"
  )
) |>
  reduce(full_join, by = "Prueba")
```


2. Evaluación de normalidad univariada

```
# Formatear y mostrar la tabla redondeando a 4 decimales
kable(
  kurtosis_normality_results_tbl,
  format = "latex",
  booktabs = TRUE,
  digits = 4,
  align = c("l", rep("c", ncol(kurtosis_normality_results_tbl) - 1))
) |>
  kable_styling(latex_options = "hold_position")
```

Tabla 2.8: Tabla de valores p de las pruebas de normalidad aplicadas a las muestras leptocúrtica, mesocúrtica y platicúrtica que generamos.

Prueba	Leptocúrtica	Mesocúrtica	Platicúrtica
Shapiro-Wilk	0	0.9409	0.0000
Kolmogorov-Smirnov	0	0.9673	0.0006
Anderson-Darling	0	0.7535	0.0000
Jarque-Bera	0	0.8298	0.0000

Nuevamente notamos que todas estas pruebas son muy sensibles a desviaciones de la normalidad para muestras grandes. En general, es difícil esperar que una muestra obtenida a partir de datos reales siga exactamente una distribución normal, por lo que hay que ser cautelosos al interpretar los resultados de estas pruebas y considerar el contexto del análisis estadístico que se esté realizando.

Capítulo 3

Escalamiento y transformaciones

El escalamiento y las transformaciones de variables aleatorias y muestras son dos maneras en las que podemos cambiar la manera en la que representamos la información contenida en un modelo estadístico. Estas técnicas son útiles en diversas situaciones, como cuando queremos comparar variables con diferentes unidades de medida o cuando buscamos mejorar la normalidad de los datos para cumplir con los supuestos de ciertos análisis estadísticos. La principal diferencia entre ambas es que el escalamiento preserva la forma de la distribución original, mientras que las transformaciones pueden alterar dicha forma así como sus propiedades estadísticas.

3.1. Escalamiento de variables aleatorias

Un problema común en el análisis multivariado es que las variables involucradas pueden tener diferentes unidades de medida o rangos de valores. Esto provoca que algunas variables dominen el análisis simplemente por su escala, lo que puede llevar a interpretaciones erróneas. En esta sección discutiremos los métodos más comunes para escalar y estandarizar variables y muestras, lo que ayuda a mitigar este tipo de problemas.

Definición 3.1.1. Una *transformación afín* de una variable aleatoria X es una transformación de la forma

$$Y = aX + b$$

donde a y b son constantes.

Comentario 3.1.2. En estadística, a menudo se habla de *escalamiento* (aunque incluya traslación) para referirse a transformaciones afines.

Suponga que X tiene media μ y desviación estándar σ . Entonces la variable

$$Y = aX + b$$

tendrá media $a\mu + b$ y desviación estándar $|a|\sigma$. En particular, si elegimos $a = 1/\sigma$ y $b = -\mu/\sigma$, entonces la variable

$$Z = \frac{X - \mu}{\sigma}$$

tendrá media 0 y desviación estándar 1. Esta transformación se conoce como la *estandarización* de la variable X y, en este caso, decimos que Z es la versión *estandarizada* de X (también conocida como su *puntaje z*).

Comentario 3.1.3. Si X no es normal, Z no será normal estándar. La estandarización no transforma una distribución no normal en una distribución normal.

Observación 3.1.4. Por definición,

$$\gamma_1(Z) = \mathbb{E}[Z^3] = \frac{\mathbb{E}[(X - \mu)^3]}{\sigma^3} = \gamma_1(X)$$

y

$$\gamma_2(Z) = \mathbb{E}[Z^4] - 3 = \frac{\mathbb{E}[(X - \mu)^4]}{\sigma^4} - 3 = \gamma_2(X).$$

Como ejemplo, consideremos de nuevo la distribución Gamma con parámetros $\alpha = 2$ y $\beta = 1$. En este caso

$$\mu = \frac{\alpha}{\beta} = 2 \quad \text{y} \quad \sigma = \frac{\sqrt{\alpha}}{\beta} = \sqrt{2}.$$

Con esta información, generamos la gráfica de la versión estandarizada de la Gamma y la comparamos con la original.

```
library(tidyverse)
library(patchwork)
library(krulRutils)
library(car)

# Parámetros de la distribución Gamma
alpha <- 2
beta <- 1

# Calculamos la media, la mediana y la desviación estándar
gamma_mean <- alpha / beta
gamma_median <- qgamma(0.5, shape = alpha, rate = beta)
gamma_sd <- sqrt(alpha) / beta

# Creamos una secuencia de valores para X
x_data <- seq(0, 8, length.out = 1000)

# Calculamos la PDF de la distribución Gamma original
gamma_distribution_tbl <- tibble(
  x_data = x_data,
  density_data = dgamma(x_data, shape = alpha, rate = beta)
)

# Calculamos la PDF de la distribución Gamma estandarizada
z_gamma_distribution_tbl <- gamma_distribution_tbl |>
```

3. Escalamiento y transformaciones

```
mutate(  
  z_data = (x_data - gamma_mean) / gamma_sd,  
  z_density = dgamma(x_data, shape = alpha, rate = beta) * gamma_sd  
) |>  
select(z_data, z_density)  
  
# Calculamos la media y mediana estandarizadas  
z_gamma_mean <- 0  
z_gamma_median <- (gamma_median - gamma_mean) / gamma_sd  
  
# Generamos la gráfica de la distribución Gamma original  
gamma_distribution_plot <- gamma_distribution_tbl |>  
ggplot(aes(x = x_data, y = density_data)) +  
  geom_line(  
    color = c_pal("C blue"),  
    linewidth = 1  
  ) +  
  geom_vline(  
    xintercept = gamma_mean,  
    color = c_pal("C red"),  
    linetype = "dashed",  
    linewidth = 1  
  ) +  
  geom_vline(  
    xintercept = gamma_median,  
    color = c_pal("C green"),  
    linetype = "dashed",  
    linewidth = 1  
  ) +  
  annotate(  
    "text",  
    x = gamma_mean,  
    y = 0.6,  
    label = "Media",  
    color = c_pal("C red"),  
    hjust = -0.1  
  ) +  
  annotate(  
    "text",  
    x = gamma_median,  
    y = 0.6,  
    label = "Mediana",  
    color = c_pal("C green"),  
    hjust = 1.1  
  )
```

```
) +  
coord_cartesian(ylim = c(0, 0.65)) +  
labs(  
  x = "X",  
  y = "Densidad"  
) +  
theme_krul()  
  
# Generamos la gráfica de la distribución Gamma estandarizada  
z_gamma_distribution_plot <- z_gamma_distribution_tbl |>  
ggplot(aes(x = z_data, y = z_density)) +  
geom_line(  
  color = c_pal("C blue"),  
  linewidth = 1  
) +  
geom_vline(  
  xintercept = z_gamma_mean,  
  color = c_pal("C red"),  
  linetype = "dashed",  
  linewidth = 1  
) +  
geom_vline(  
  xintercept = z_gamma_median,  
  color = c_pal("C green"),  
  linetype = "dashed",  
  linewidth = 1  
) +  
annotate(  
  "text",  
  x = z_gamma_mean,  
  y = 0.6,  
  label = "Media",  
  color = c_pal("C red"),  
  hjust = -0.1  
) +  
annotate(  
  "text",  
  x = z_gamma_median,  
  y = 0.6,  
  label = "Mediana",  
  color = c_pal("C green"),  
  hjust = 1.1  
) +  
coord_cartesian(ylim = c(0, 0.65)) +
```

3. Escalamiento y transformaciones

```
labs(  
  title = paste(  
    "Estandarizada"  
  ),  
  x = "X",  
  y = "Densidad"  
) +  
theme_krul()  
  
gamma_distribution_plot <- gamma_distribution_plot +  
  labs(  
    title = "Original"  
  )  
  
combined_gamma_plot <- gamma_distribution_plot + z_gamma_distribution_plot +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Distribución Gamma: original y estandarizada",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
  
combined_gamma_plot
```

Distribución Gamma: original y estandarizada

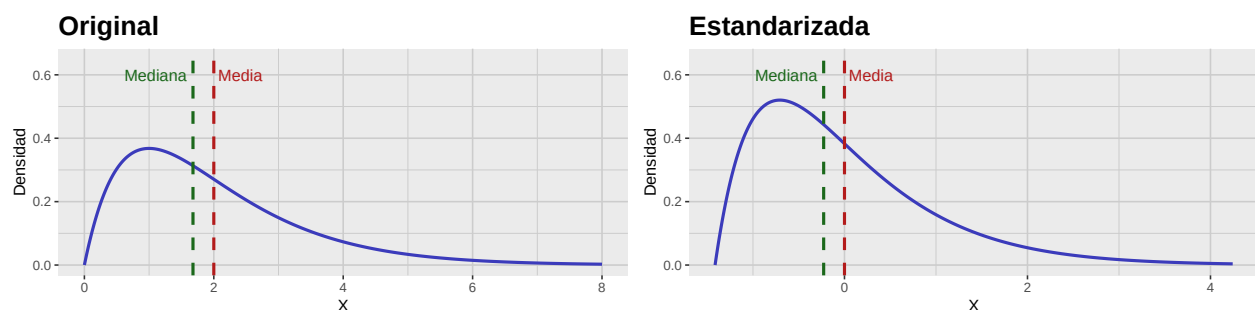


Figura 3.1: Comparación de la función de densidad (PDF) de la distribución Gamma original y su versión estandarizada.

3.2. Escalamiento de muestras

Para una muestra dada, no siempre se conoce la distribución subyacente; en particular, pueden desconocerse la media y la desviación poblacionales. Aun así, podemos escalar los datos; hay varios métodos comunes:

1. *Estandarización*: Usa la media y desviación muestrales. La muestra estandarizada es

$$T = \frac{X - \bar{X}}{S},$$

donde $\bar{X}$ es la media muestral y S la desviación muestral.

2. *Escalamiento Min-Max*: Escala a un rango fijo, usualmente $[0,1]$. La muestra escalada es

$$T = \frac{X - \min(X)}{\max(X) - \min(X)},$$

donde $\min(X)$ y $\max(X)$ son el mínimo y máximo de la muestra. Es muy sensible a atípicos al usar extremos.

3. *Escalamiento robusto*: Usa la mediana y el rango intercuartil (IQR). La muestra escalada es

$$T = \frac{X - \text{median}(X)}{\text{IQR}(X)},$$

donde $\text{IQR}(X) = Q_3 - Q_1$. Es más robusto a atípicos que Min-Max.

4. *Normalización*: Divide cada dato por la norma del vector. La muestra normalizada es

$$T = \frac{X}{\|X\|},$$

donde $\|X\|$ es la norma. Se usa con menor frecuencia que los anteriores.

Comentario 3.2.1. Como se mencionó, escalar no vuelve normal una distribución no normal. En particular, la asimetría y la curtosis en exceso muestrales no cambian.

Como ejemplo, consideraremos los distintos métodos de escalado aplicados a la muestra de la distribución Gamma que generamos anteriormente.

```
scale_lookup_tbl <- tibble(  
  code = c(  
    "sample",  
    "sample_standard",  
    "sample_min_max",  
    "sample_robust",  
    "sample_normal"  
  ),  
  label = c(  
    "Original Sample",  
    "Estandarizada",
```


3. Escalamiento y transformaciones

```
"Min-Max Scaled",
"Robust Scaled",
"Normalized"
)
)

# Tamaño de la muestra
n <- 300

gamma_sample <- rgamma(n, shape = alpha, rate = beta)

gamma_sample_tbl <- tibble(
  sample = gamma_sample
)

scaled_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    sample_standard = standard_scale(sample),
    sample_min_max = min_max_scale(sample),
    sample_robust = robust_scale(sample),
    sample_normal = normal_scale(sample)
  ) |>
  pivot_longer(
    cols = everything(),
    names_to = "scaling_method",
    values_to = "sample_scaled"
  ) |>
  convert_codes_to_factor(
    code_col = scaling_method,
    lookup_tbl = scale_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = scaling_method_factor
  )

scaled_gamma_qq_plot <- scaled_gamma_sample_tbl |>
  ggplot(aes(sample = sample_scaled)) +
  facet_wrap(
    ~scaling_method_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_pal("C red"),
    size = 0.3
  )
```

```
) +
geom_qq_line(
  color = c_pal("C blue"),
  linewidth = 0.5
) +
labs(
  title = "Gráficos QQ",
  x = "Cuantiles teóricos",
  y = "Cuantiles muestrales"
) +
theme_krul()

scaled_gamma_histogram <- scaled_gamma_sample_tbl |>
ggplot(aes(x = sample_scaled)) +
facet_wrap(
  ~scaling_method_factor,
  nrow = 5,
  scales = "free"
) +
geom_histogram(
  bins = 30,
  fill = c_pal("C blue"),
  color = "black",
  alpha = 0.7
) +
labs(
  title = "Histogramas",
  x = "Valores muestrales",
  y = "Densidad"
) +
theme_krul()

scaled_gamma_plot <- scaled_gamma_qq_plot + scaled_gamma_histogram +
plot_layout(ncol = 2) +
plot_annotation(
  title = "Comparación de métodos de escalamiento aplicados a la muestra Gamma",
  theme = theme(
    plot.title = element_text(
      size = 24,
      face = "bold"
    )
  )
)

scaled_gamma_plot
```

3. Escalamiento y transformaciones

Comparación de métodos de escalamiento aplicados a la muestra Gamma

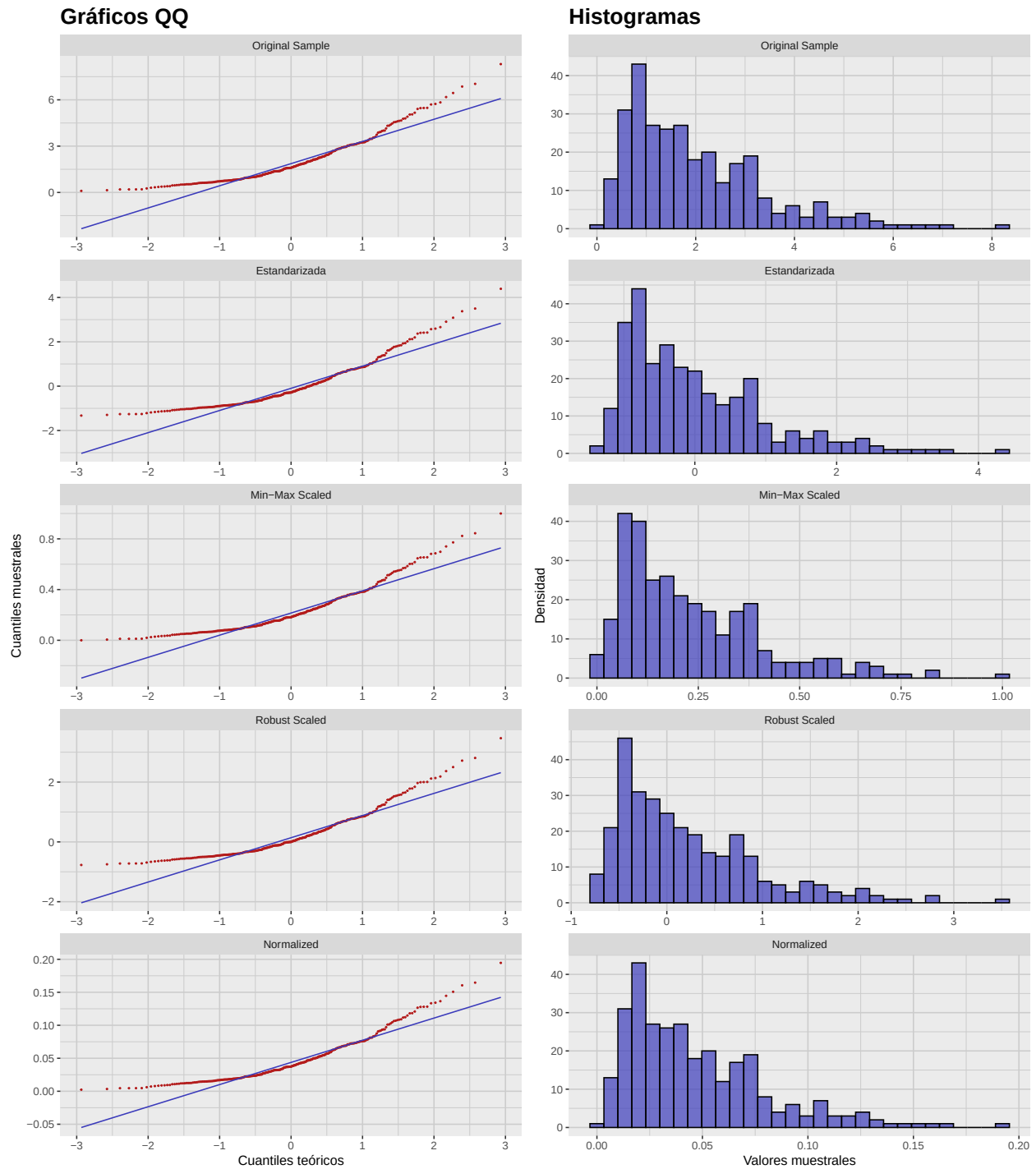


Figura 3.2: Comparación de distintos métodos de escalamiento aplicados a la muestra Gamma. Note que los QQ-plots son versiones reescaladas del gráfico QQ original y, por tanto, tienen la misma forma.

3.3. Transformaciones

Para acercar los datos a la normalidad, pueden aplicarse transformaciones. Algunas comunes son:

1. *Transformación de Box-Cox*: familia de transformaciones potenciadas aplicables sólo a datos positivos. Está dada por:

$$\tilde{X} = \begin{cases} \frac{X^\lambda - 1}{\lambda} & \text{if } \lambda \neq 0 \\ \log(X) & \text{if } \lambda = 0 \end{cases}$$

donde λ es un parámetro.

2. *Transformación de Yeo-Johnson*: similar a Box-Cox, pero admite valores negativos; útil con asimetrías positivas o negativas. Está dada por:

$$\tilde{X} = \begin{cases} \frac{((X+1)^\lambda - 1)}{\lambda} & \text{if } X \geq 0, \lambda \neq 0 \\ \frac{-((-X+1)^{2-\lambda} - 1)}{2-\lambda} & \text{if } X < 0, \lambda \neq 2 \\ \log(X+1) & \text{if } X \geq 0, \lambda = 0 \\ \log(-X+1) & \text{if } X < 0, \lambda = 2 \end{cases}$$

donde λ es un parámetro.

Aplicaremos la transformación de Box-Cox a la muestra Gamma con $\lambda = 0, 0.25, 0.5, 0.75$ y 1 :

```
set.seed(123) # for reproducibility
# Sample size
n <- 300
b <- 1 / sqrt(2)
mu <- 0

# Generate random samples from the Gamma distribution
gamma_sample <- rgamma(n, shape = alpha, rate = beta)
gamma_sample_tbl <- tibble(
  sample = gamma_sample
)
# Generate uniform random numbers in (0,1)
u <- runif(n)
# Apply inverse CDF of Laplace
laplace_sample <- ifelse(u < 0.5,
  mu + b * log(2 * u),
  mu - b * log(2 * (1 - u))
)
laplace_sample_tbl <- tibble(
  sample = laplace_sample
)

box_cox_lookup_tbl <- tibble(
  code = c(
    "box_cox_0",
```

3. Escalamiento y transformaciones

```
    "box_cox_025",
    "box_cox_05",
    "box_cox_075",
    "box_cox_1"
  ),
  label = c(
    "lambda = 0",
    "lambda = 0.25",
    "lambda = 0.5",
    "lambda = 0.75",
    "lambda = 1"
  ),
)

box_cox_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    box_cox_0 = bcPower(sample, lambda = 0),
    box_cox_025 = bcPower(sample, lambda = 0.25),
    box_cox_05 = bcPower(sample, lambda = 0.5),
    box_cox_075 = bcPower(sample, lambda = 0.75),
    box_cox_1 = bcPower(sample, lambda = 1)
  ) |>
  pivot_longer(
    starts_with("box_cox_"),
    names_to = "lambda",
    values_to = "transformed_sample"
  ) |>
  convert_codes_to_factor(
    code_col = lambda,
    lookup_tbl = box_cox_lookup_tbl,
    lookup_code_col = code,
    lookup_label_col = label,
    new_factor_col_name = lambda_factor
  )

box_cox_gamma_qq_plot <- box_cox_gamma_sample_tbl |>
  ggplot(aes(sample = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_qq(
    color = c_palette["C red"],
```

```
      size = 0.3
    ) +
    geom_qq_line(
      color = c_palette["C blue"],
      linewidth = 0.5
    ) +
    labs(
      title = "Gráficos QQ",
      x = "Cuantiles teóricos",
      y = "Cuantiles muestrales"
    ) +
    theme_krul()

box_cox_gamma_histogram <- box_cox_gamma_sample_tbl |>
  ggplot(aes(x = transformed_sample)) +
  facet_wrap(
    ~lambda_factor,
    nrow = 5,
    scales = "free"
  ) +
  geom_histogram(
    bins = 30,
    fill = c_palette["C blue"],
    color = "black",
    alpha = 0.7
  ) +
  labs(
    title = "Histogramas",
    x = "Valores muestrales",
    y = "Densidad"
  ) +
  theme_krul()

box_cox_gamma_plot <- box_cox_gamma_qq_plot + box_cox_gamma_histogram +
  plot_layout(ncol = 2) +
  plot_annotation(
    title = "Transformaciones Box-Cox con distintos valores de lambda",
    theme = theme(
      plot.title = element_text(
        size = 24,
        face = "bold"
      )
    )
  )
)
```

3. Escalamiento y transformaciones

box_cox_gamma_plot

Transformaciones Box-Cox con distintos valores de lambda

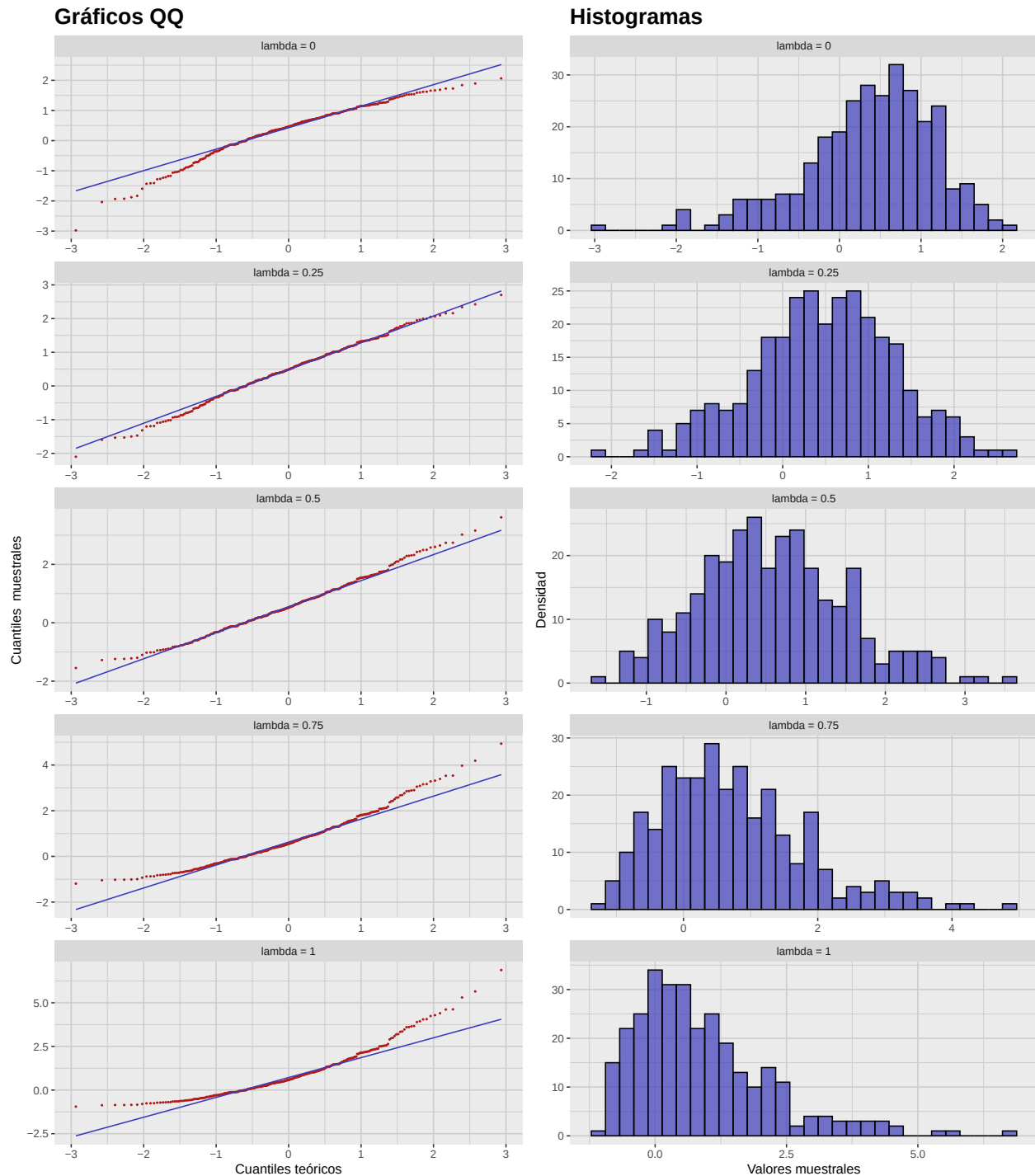


Figura 3.3: Comparación de distintas transformaciones de Box-Cox aplicadas a la muestra Gamma, con parámetro $\lambda = 0, 0.25, 0.5, 0.75$ y 1 .

3.4. Optimización de normalidad (Box–Cox)

Como vimos, la transformación de Box–Cox puede acercar los datos a la normalidad. La elección de λ es crucial: buscamos el valor que maximiza la log-verosimilitud. Esto puede realizarse con `car::powerTransform`, que estima el λ óptimo. La figura muestra el perfil de log-verosimilitud para la muestra Gamma:

```
# Intercept-only model
fit <- lm(sample ~ 1, data = gamma_sample_tbl)

# Estimate lambda
bc <- powerTransform(fit)

# Extract log-likelihood profile
bc_prof <- boxCox(fit, lambda = seq(-2, 2, 0.1), plotit = FALSE)

# Convert to tibble for ggplot
df <- tibble(lambda = bc_prof$x, loglik = bc_prof$y)

ggplot(df, aes(x = lambda, y = loglik)) +
  geom_line(color = c_pal("C blue"), linewidth = 1) +
  geom_vline(xintercept = bc$lambda, linetype = "dashed", color = c_pal("C red")) +
  labs(
    title = "Transformación de Box-Cox",
    x = expression(lambda),
    y = "Log-verosimilitud"
  ) +
  theme_krul()
```

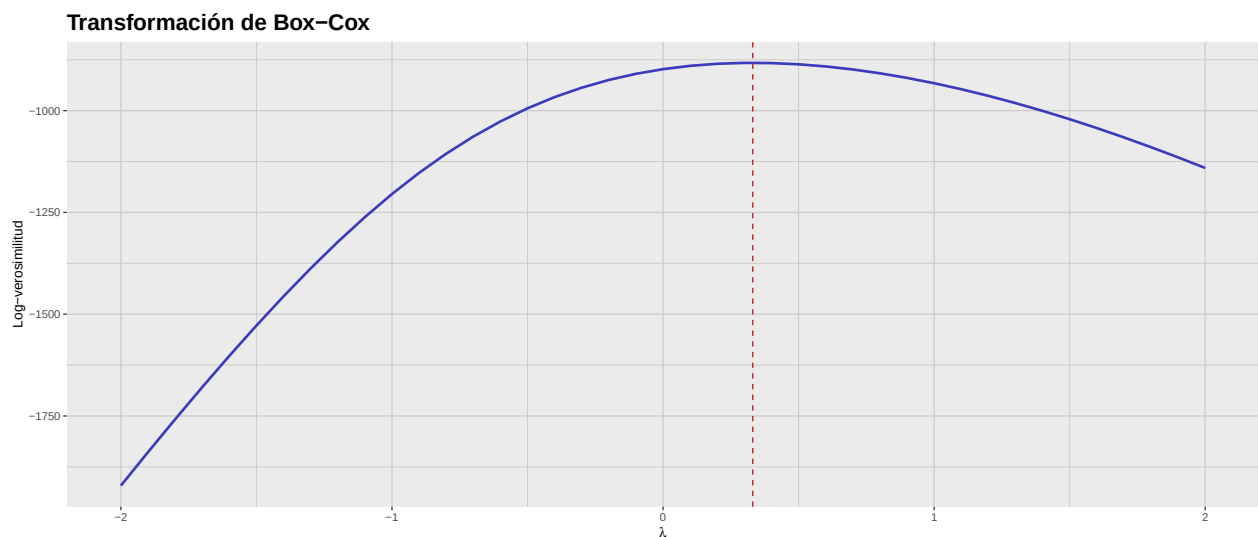


Figura 3.4: Perfil de log-verosimilitud para la transformación de Box–Cox aplicada a la muestra Gamma. La línea discontinua indica el valor óptimo de lambda.

3. Escalamiento y transformaciones

Finalmente, extraemos el λ óptimo y transformamos los datos con ese valor:

```
optimal_lambda <- bc$lambda

optimal_gamma_sample_tbl <- gamma_sample_tbl |>
  mutate(
    transformed_sample = bcPower(sample, lambda = optimal_lambda)
  )

compute_normality_pvalues(
  data = optimal_gamma_sample_tbl,
  value_col = transformed_sample,
  sample_name = "p-value"
)
```

```
# A tibble: 4 x 2
  Prueba      `p-value`
  <fct>      <dbl>
1 Shapiro-Wilk      0.974
2 Kolmogorov-Smirnov 1.000
3 Anderson-Darling   0.977
4 Jarque-Bera        0.878
```

Como se observa, la transformación de Box–Cox con el valor óptimo de λ mejora significativamente la normalidad de los datos, pues todas las pruebas arrojan valores p altos. A continuación se presentan el gráfico QQ y el histograma de los datos transformados:

```
optimal_gamma_qq_plot <- optimal_gamma_sample_tbl |>
  ggplot(aes(sample = transformed_sample)) +
  geom_qq(
    color = c_palette["C red"],
    size = 0.3
  ) +
  geom_qq_line(
    color = c_palette["C blue"],
    linewidth = 0.5
  ) +
  labs(
    title = "Gráfico QQ",
    x = "Cuantiles teóricos",
    y = "Cuantiles muestrales"
  ) +
  theme_krul()

optimal_gamma_histogram <- optimal_gamma_sample_tbl |>
  ggplot(aes(x = transformed_sample)) +
```

```
geom_histogram(  
  bins = 30,  
  fill = c_palette["C blue"],  
  color = "black",  
  alpha = 0.7  
) +  
labs(  
  title = "Histograma",  
  x = "Valores muestrales",  
  y = "Densidad"  
) +  
theme_krul()  
  
optimal_gamma_plot <- optimal_gamma_qq_plot + optimal_gamma_histogram +  
  plot_layout(ncol = 2) +  
  plot_annotation(  
    title = "Transformación Gamma óptima",  
    theme = theme(  
      plot.title = element_text(  
        size = 24,  
        face = "bold"  
      )  
    )  
  )  
  
optimal_gamma_plot
```

3. Escalamiento y transformaciones

Transformación Gamma óptima

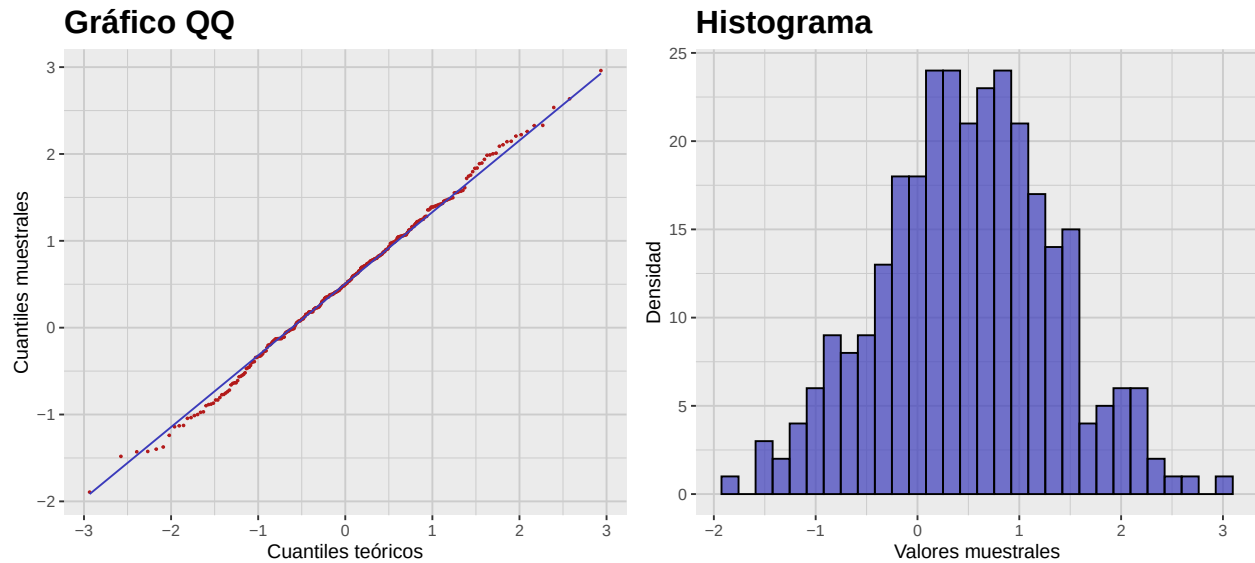


Figura 3.5: Gráfico QQ e histograma de la transformación Gamma óptima. Nótese el mejor ajuste a la normalidad en comparación con la muestra Gamma original.

3.5. Actividad: Visualizando la no normalidad de muestras

Para cada una de las siguientes distribuciones, construya una muestra de tamaño 300 y calcule la asimetría y curtosis muestrales. Después, construya y compare sus gráficos QQ e histogramas.

1. Distribución exponencial con tasa $\lambda = 1$.
2. Distribución uniforme en $[0, 1]$.
3. Distribución beta con $\alpha = 2$ y $\beta = 5$.
4. Distribución de Cauchy con localización $x_0 = 0$ y escala $\gamma = 1$.
5. Distribución χ_k^2 con $k = 3$ grados de libertad.

